

Learnware for CSI Feedback: Scene-specific Small Models Can Do Big

Xiangyi Li, *Graduate Student Member, IEEE*, Jiajia Guo, *Member, IEEE*, Chao-Kai Wen, *Fellow, IEEE*, Xin Geng, *Senior Member, IEEE*, Shi Jin, *Fellow, IEEE*, and Zhi-Hua Zhou, *Fellow, IEEE*

Abstract—Intelligent channel state information (CSI) feedback is essential for realizing the high capacity and spectral efficiency goals of future 6G systems, yet existing deep learning solutions face a trade-off between model generalization and scenario-specific performance. Large neural networks generalize well but incur high computational and tuning costs, while small models excel in particular environments but require repetitive costly end-to-end training for each base station (BS). To address these challenges, we introduce a model repository-based deployment framework in which a centralized AI data center maintains a catalog of scene-specific CSI models. The repository is enhanced with a Learnware-based framework, where each model is associated with a *specification* including semantic part (network architecture parameters) and statistical part (codebook-fingerprint embeddings of training-data distributions). A BS submits only its local statistical specifications to retrieve the most relevant pre-trained model, enhancing data privacy by avoiding raw CSI transmission and drastically reducing retrieval latency and communication overhead. We further develop a data-driven search strategy that matches codebook fingerprints to model performance, achieving over 90% selection accuracy. In simulations, our scheme yields 18.8% and 57.7% performance improvements over the General Model in LOS and NLOS scenarios, respectively while reducing local fine-tuning by up to 1000 samples and 100 epochs. This Learnware-based approach minimizes redundant training, maximizes model reuse, and supports rapid, privacy-enhancing deployment of CSI feedback models.

Index Terms—Massive MIMO, CSI feedback, deep learning, model repository, model selection.

I. INTRODUCTION

ACCORDING to the latest 6G vision released by the International Telecommunication Union (ITU) [1], future wireless communication systems will demand higher throughput, lower latency, greater reliability, massive connectivity, and more efficient spectrum utilization [2], [3]. Massive multiple-input multiple-output (MIMO) arrays are essential to achieving these goals, offering spatial multiplexing gains that improve

X. Li, J. Guo, and S. Jin are with the National Mobile Communications Research Laboratory, Southeast University, Nanjing, 210096, China (e-mail: Xiangyi_li@seu.edu.cn, jiajiagu@seu.edu.cn, jinshi@seu.edu.cn). J. Guo is now with the Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology.

C.-K. Wen is with the Institute of Communications Engineering, National Sun Yat-sen University, Kaohsiung 80424, Taiwan (e-mail: chaokai.wen@mail.nsysu.edu.tw).

X. Geng is with the School of Computer Science and Engineering and the Key Laboratory of New Generation Artificial Intelligence Technology and Its Interdisciplinary Applications (Southeast University), Ministry of Education, Southeast University, Nanjing 211189, China (e-mail: xgeng@seu.edu.cn).

Z.-H. Zhou is with the National Key Laboratory for Novel Software Technology, and also with the School of Artificial Intelligence, Nanjing University, Nanjing 210023, China (e-mail: zhouzh@lamda.nju.edu.cn).

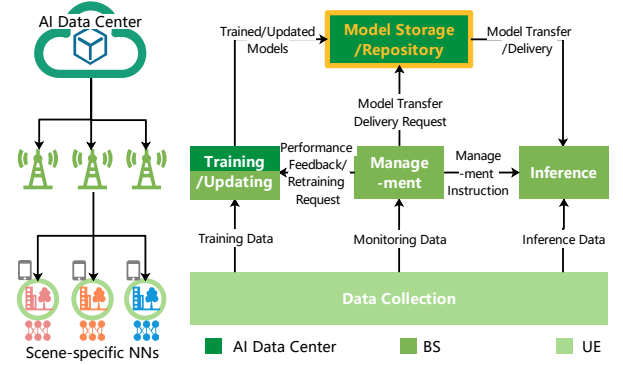


Fig. 1. Model repository-based CSI feedback deployment, where the respective functions of the AI data center, BS and UE are marked in different colors.

both capacity and spectral efficiency [4]. In frequency division duplexing (FDD) MIMO systems, user equipment (UE) estimates the downlink channel state information (CSI) and feeds it back to the base station (BS) for precoding. Since transmitting full CSI entails substantial overhead, efficient compression and reconstruction mechanisms are critical.

Conventional CSI compression techniques, such as compressed sensing [5] and codebook-based methods [6], provide theoretical guarantees but often underperform in complex, large-scale deployments [7]. Deep learning (DL)-based methods have emerged as effective alternatives by modeling CSI matrices as high-dimensional images and training autoencoders for end-to-end compression and reconstruction. This approach was first demonstrated in CsiNet [8], which achieved significant reductions in feedback overhead while maintaining reconstruction accuracy. Subsequent works have incorporated convolutional structures, attention mechanisms, and lightweight network designs to further improve performance and reduce inference latency [9]–[15]. In parallel with these research advances, the 3rd Generation Partnership Project (3GPP) initiated AI-driven CSI feedback as a study item in Release 18 (R18) and subsequently advanced it to a work item in Release 20 [16], [17]. More broadly, AI has been recognized as a key enabler of 6G and is being incorporated into air interface design across multiple 3GPP working groups [18]–[20].

Despite these advances, practical deployment of AI-based CSI feedback remains challenging. Current systems typically centralize model training in AI data centers, which either generate personalized models based on CSI uploaded from BSs or construct unified models trained on aggregated data [21]–[23]. Unified models facilitate broad applicability but may exhibit suboptimal performance in specialized environments,

while also incurring high computational costs, long training times, and greater reliance on expert tuning. In contrast, scene-specific small models trained on localized CSI can achieve near-optimal performance in their target scenarios, supporting dynamic model switching or rapid fine-tuning during deployment [24]–[27]. For example, the 3GPP R18 *Model Switch* strategy introduces multiple predefined configurations (e.g., 38.901 UMi/UMa) for online selection [16], [28]. However, models trained for personalized deployment are typically discarded after use, resulting in wasted computational resources and storage. More importantly, existing literature predominantly focuses on model training and inference, with limited attention to systematic model storage, management, and reuse across deployments, which constitutes a critical gap that this work aims to address.

To address these limitations, we propose a model repository-based deployment framework, illustrated in Fig. 1, which aligns with the 3GPP AI/ML lifecycle management framework [20]. Our framework focuses primarily on model storage and repository, designing its effective connection with other modules. A centralized repository maintains a catalog of scene-specific AI models, each annotated with standardized metadata. In management, when a BS submits a Delivery Request (retrieval request) by providing a small amount of local CSI or a brief scene description, the repository returns the most appropriate pre-trained model with model transfer for the BS's inference and further model updating, minimizing the need for local fine-tuning and avoiding redundant training.

Efficient model retrieval requires informative yet privacy-aware metadata. To this end, we adopt and tailor the Learnware paradigm to CSI feedback deployment. Learnware is defined as Model plus Specification [29]–[32], where each model is accompanied by a specification that characterizes its capability without exposing the training data. In this work, we extend the Learnware specification framework to the CSI compression task. Each specification consists of two components. The semantic specification encodes architectural attributes such as input and output dimensions, compression ratios, and task types. The statistical specification summarizes the training data distribution through compact embeddings derived from codebook fingerprint frequency vectors. This design enables efficient model comparison and retrieval without sharing raw CSI. It has been shown in [33] that the specification mechanism in [29], [31] protects the original training data even against inference and linkage attacks under strong adversarial settings. Therefore, by uploading only statistical specifications rather than raw CSI, BSs avoid exposing raw channel data, mitigating a primary privacy risk, while enabling rapid model retrieval with low communication overhead. The main contributions of this work are summarized as follows:

- **First repository-centric CSI feedback deployment architecture.** We propose a principled model repository-based deployment framework for AI-driven CSI feedback, aligned with the 3GPP AI and ML lifecycle management architecture. Unlike conventional retrain-and-discard paradigms, the proposed framework enables systematic model reuse and adaptation across heterogeneous deployment scenarios. This architecture fundamentally

transforms CSI feedback from isolated model training to lifecycle-aware model orchestration, substantially reducing redundant training cost and accelerating deployment.

- **Learnware-enabled privacy-enhancing CSI model indexing.** We introduce a Learnware-based repository design tailored to CSI compression tasks and develop a novel codebook fingerprint **pdf** as the statistical specification. The proposed representation captures angular-domain channel structure through DFT beamspace projection while avoiding raw CSI exposure. This design establishes a unified, privacy-enhancing, and computation-efficient model indexing mechanism that enables scalable model comparison across diverse radio environments.
- **Scalable sub-millisecond retrieval with controllable accuracy-latency tradeoff.** We develop a high-precision and low-latency model retrieval framework based on **pdf** similarity matching, achieving over 90% selection accuracy. To meet stringent CSI feedback timing constraints, we further design a multi-level locality-sensitive hashing (LSH) structure that scales to billion-model repositories and enables 0.25–1.15 ms retrieval latency. The proposed mechanism provides a controllable tradeoff between retrieval accuracy and latency through adjustable search depth, making it suitable for real-time wireless systems.

The remainder of this paper is organized as follows. Section II describes the system and channel models. Section III introduces the Learnware-based repository structure and the specification designs. Section IV displays the online deployment framework with Learnware repository. Section V presents simulation settings, comparative evaluations, and ablation studies. Section VI concludes the paper and outlines future work.

II. SYSTEM MODEL

A. Massive MIMO FDD System

Consider a downlink massive MIMO system where a BS equipped with N_t uniform linear array (ULA) antennas serves a UE with N_r antennas. The received signal at the UE is given by [7], [8]:

$$\mathbf{y} = \mathbf{H}\mathbf{v}s + \mathbf{n}, \quad (1)$$

where $\mathbf{H} \in \mathbb{C}^{N_r \times N_t}$ is the channel matrix, $\mathbf{v} \in \mathbb{C}^{N_t}$ is the precoding vector with $\|\mathbf{v}\|_2 = 1$, $s \in \mathbb{C}$ is the transmitted symbol with $\mathbb{E}[|s|^2] = 1$, and $\mathbf{n} \in \mathbb{C}^{N_r}$ is the additive white Gaussian noise with $\mathbf{n} \sim \mathcal{CN}(0, \sigma^2 \mathbf{I}_{N_r})$. The channel matrix can be decomposed via singular value decomposition (SVD) as:

$$\mathbf{H} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^H, \quad (2)$$

where $\mathbf{U} \in \mathbb{C}^{N_r \times N_r}$ and $\mathbf{V} \in \mathbb{C}^{N_t \times N_t}$ are unitary matrices, and $\mathbf{\Sigma} \in \mathbb{R}^{N_r \times N_t}$ is a diagonal matrix containing the singular values in descending order. For single-stream transmission, the BS selects the first column of $\mathbf{V}$, corresponding to the largest singular value, as the precoding vector $\mathbf{v}$ [34].

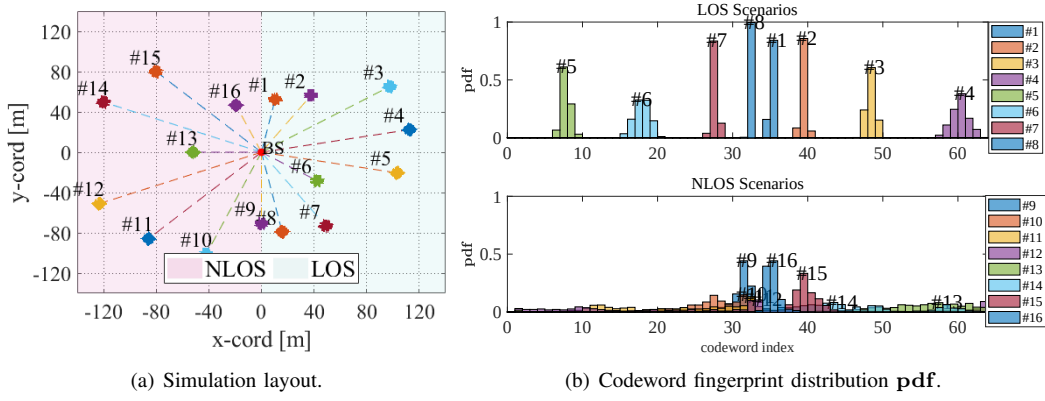


Fig. 2. Sample frequency distribution of codeword fingerprints. (a) Layout of QuaDRiGa-simulated scenarios with UEs randomly distributed in $5\text{ m} \times 5\text{ m}$ rectangle subregions. (b) Corresponding codeword fingerprint distributions **pdf** for LOS and NLOS conditions.

B. DL-Based CSI Feedback

The DL-based CSI feedback framework adopts an autoencoder architecture. The input to the network is the precoding vector $\mathbf{v}$, which is preprocessed by separating its real and imaginary parts and stacking them into a matrix of shape $(2, N_t)$. The matrix is then normalized to the range $[0, 1]$ and still denoted as $\mathbf{v}$.

The autoencoder comprises an encoder $\text{enc}(\cdot)$ and a decoder $\text{dec}(\cdot)$, deployed at the UE and BS, respectively. The feedback bitstream is obtained through encoder compression followed by a uniform quantizer f_Q with quantization level Q :

$$\mathbf{s}_Q = f_Q(\text{enc}(\mathbf{v}; \boldsymbol{\Omega}_{\text{enc}})), \quad (3)$$

where $\boldsymbol{\Omega}_{\text{enc}}$ denotes the encoder network parameters. The M -length bitstream $\mathbf{s}_Q$, with compression ratio $\text{CR} = M/(2Q \times N_t)$. At the BS, the decoder reconstructs the precoding vector:

$$\hat{\mathbf{v}} = \text{dec}(f_Q^{-1}(\mathbf{s}_Q); \boldsymbol{\Omega}_{\text{dec}}), \quad (4)$$

where $\boldsymbol{\Omega}_{\text{dec}}$ are the decoder parameters.

To evaluate reconstruction fidelity, we adopt the squared generalized cosine similarity (SGCS) metric:

$$\rho^2 = \frac{1}{|\mathcal{D}|} \sum_{\mathbf{v} \in \mathcal{D}} \left(\frac{|\mathbf{v}^H \hat{\mathbf{v}}|}{\|\mathbf{v}\|_2 \|\hat{\mathbf{v}}\|_2} \right)^2. \quad (5)$$

During end-to-end training, we minimize the cosine distance between the original and reconstructed precoding vectors:

$$\mathcal{L} = \frac{1}{|\mathcal{D}|} \sum_{\mathbf{v} \in \mathcal{D}} \left(1 - \frac{|\mathbf{v}^H \hat{\mathbf{v}}|}{\|\mathbf{v}\|_2 \|\hat{\mathbf{v}}\|_2} \right), \quad (6)$$

where $\mathcal{D}$ denotes the training dataset and $|\mathcal{D}|$ its cardinality. This loss function is directly aligned with the SGCS metric, thereby ensuring consistency between the training objective and the evaluation criterion.

C. Codeword Fingerprint

To characterize the statistical properties of CSI datasets, we adopt a codebook fingerprinting technique based on [35]. We utilize a discrete Fourier transform (DFT) codebook, which is well-suited to uniform linear arrays and captures long-term angular characteristics of wireless channels [36].

The DFT codebook of size $N_{\text{code}} = 2^B$ is defined as $\mathbf{W} = [\mathbf{w}_0, \mathbf{w}_1, \dots, \mathbf{w}_{N_{\text{code}}-1}]$, where the n -th codeword is given by:

$$\mathbf{w}_n = \frac{1}{\sqrt{N_t}} \left[1, e^{j \frac{2\pi}{N_{\text{code}}} n}, \dots, e^{j \frac{2\pi}{N_{\text{code}}} (N_t-1)n} \right]^T. \quad (7)$$

For a given precoding vector $\mathbf{v}$, its fingerprint in the angular domain is computed as:

$$\mathbf{f} = \mathbf{v}^H \mathbf{W} = [f_1, f_2, \dots, f_{N_{\text{code}}}]. \quad (8)$$

The index of the dominant beam direction is:

$$\text{idx} = \underset{i \in \{1, \dots, N_{\text{code}}\}}{\text{argmax}} |f_i|. \quad (9)$$

By aggregating the dominant directions over the dataset, we compute a histogram to approximate the angular probability density function (PDF):

$$\mathbf{pdf} = \text{hist}_{\mathcal{D}}(\text{idx}). \quad (10)$$

This statistical distribution **pdf** reflects the angular spread of energy in the dataset, revealing environmental characteristics. In line-of-sight (LOS) cases, the dominant direction corresponds to the angle of departure (AoD) of the LOS path. In non-line-of-sight (NLOS) scenarios, it reflects the AoD of the strongest scattering path.

We illustrate this with CSI datasets generated using QuaDRiGa under the “3GPP-38.901-UMi-LOS/NLOS” settings. UEs are randomly distributed within $5\text{ m} \times 5\text{ m}$ rectangle subregions, as shown in Fig. 2(a). The resulting **pdf** distributions are presented in Fig. 2(b), where LOS scenarios exhibit more concentrated angular energy, while NLOS cases show broader dispersion due to multipath scattering. The LOS case illustrates an extreme example where small sampling areas (5 m) and ULA configuration create concentrated angular distributions. This apparent correlation is an artifact of experimental design: expanding sampling areas yields diffuse distributions, different antenna arrays break the one-to-one mapping, and each dataset restarts obstacle layouts across independent maps. Unlike CSI-positioning with fixed environments, our framework prevents inferring the precise UE position from **pdf**.

III. LEARNWARE FRAMEWORK

This section presents our Learnware-based CSI-feedback model repository. We first motivate the use of scene-specific small models and the integration of a centralized repository into the CSI-feedback workflow. We then outline the Learnware paradigm and its benefits for CSI-feedback applications.

A. Motivation

Deploying AI-driven CSI-feedback models entails two primary challenges: (1) balancing generalization with per-scene performance, and (2) managing the computational and deployment costs of large models. Our Learnware Model Repository addresses these by enabling efficient reuse of scene-specific models identified via statistical specifications. We highlight three key insights: the efficacy of small models, the power of statistical specifications, and the accelerated fine-tuning convergence they afford.

1) *Scene-Specific Small Models vs. General-Purpose Models*: Training a single large-capacity model to generalize across diverse scenarios simplifies management but demands extensive compute, memory, and often yields suboptimal performance in specific environments. In contrast, scene-specific small models, trained on localized CSI data, capture fine-grained propagation characteristics, require fewer resources, and consistently outperform general models within their target domains (see Table VI). Detailed experiments appear in Section V. This phenomenon is theoretically grounded in multi-task learning theory [37], [38]. A general model trained on heterogeneous scenes suffers from negative transfer: parameter sharing induces gradient interference across tasks, degrading performance when tasks are insufficiently related. Scene-specific models avoid this interference by dedicating capacity to a single distribution, yielding superior performance under capacity-limited regimes.

2) *Statistical Specifications vs. Semantic Specifications*: Accurate model retrieval requires more than coarse structural descriptors (e.g., network depth, codeword length). Semantic specifications alone cannot distinguish differences in data distributions due to environmental variations. Instead, we use *statistical specifications*, continuous descriptors of the training-data distribution (e.g., codeword fingerprint histograms), to index models in a continuous specification space. This approach generalizes to unseen environments and avoids the inefficiency of exhaustive validation.

3) *Rapid Fine-Tuning Convergence*: Retrieved models often require local fine-tuning. The closer a model's original training distribution is to the target domain, the fewer iterations and samples needed for convergence [39]. Fig. 3 illustrates convergence trajectories from (1) a mismatched model, (2) a general model, and (3) the nearest Learnware model. A densely populated repository ensures rapid adaptation or even direct deployment without further training.

B. Learnware Paradigm and Specification Design

The motivations described in the previous subsection support our design of a Learnware-based repository that emphasizes scene-specific model reuse, retrieval using statistical

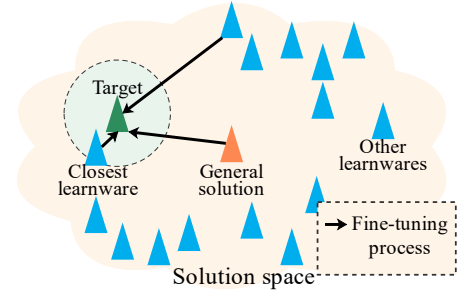


Fig. 3. CSI feedback model solution space. Curves indicate convergence paths from (1) a mismatched model, (2) a general model, and (3) the closest Learnware model.

TABLE I
COMPARISON OF PARADIGMS

Paradigm	Resource requirements	Transmission cost	Lifelong learning	Robustness to forgetting	Data privacy
Aggregate datasets	High	High	×	×	×
Learnware Paradigm	Low	Low	✓	✓	✓

specifications, and efficient adaptation through fine-tuning. Fig. 4 illustrates the framework of the Learnware-based repository, which consists mainly of two stages: *submission* and *deployment*. In the submission stage, developers package each model with its semantic and statistical specifications and upload it to the repository. In the deployment stage, users submit their task requirements, and the system matches semantic tags and locates the appropriate *specification* for model retrieval.

The Learnware paradigm [29], [31] provides a marketplace of pre-trained models that users can reuse without sharing raw data. Each Learnware entry contains a model and its specification, allowing the system to match user requests while keeping each contributor's training data local. This paradigm is provided as a heterogeneous and evolving alternative to the traditional approach of training strong generalized models from aggregated datasets without collecting their training data. Instead of learning from a large centralized dataset [40], it accumulates knowledge through an ever-growing collection of distributed models. This model-centric approach supports on-demand access to task-relevant knowledge while avoiding catastrophic forgetting and the scalability limitations of data aggregation, as depicted in Tab. I. It also offers the future possibility of reassembling multiple models to address complicated or even unplanned tasks [29].

The remainder of this section describes the submission stage and the design of specifications. The deployment stage will be discussed in the next section.

1) *Model Submission*: The Learnware system supports large-scale operation, allowing many developers to contribute models, thus creating a rich, diverse repository. Future users submit requirements to the Learnware Dock system, which identifies or reassembles learnwares. Users can then reuse these models or refine them with their data. Although BSs are encouraged to contribute high-quality models voluntarily, most CSI-feedback models are trained centrally in an AI resource data center, which can maintain a broad collection of pre-trained models. Each model is stored as a Learnware entry that includes its specifications, describing functional characteristics, enabling efficient identification and retrieval without retaining the original training data.

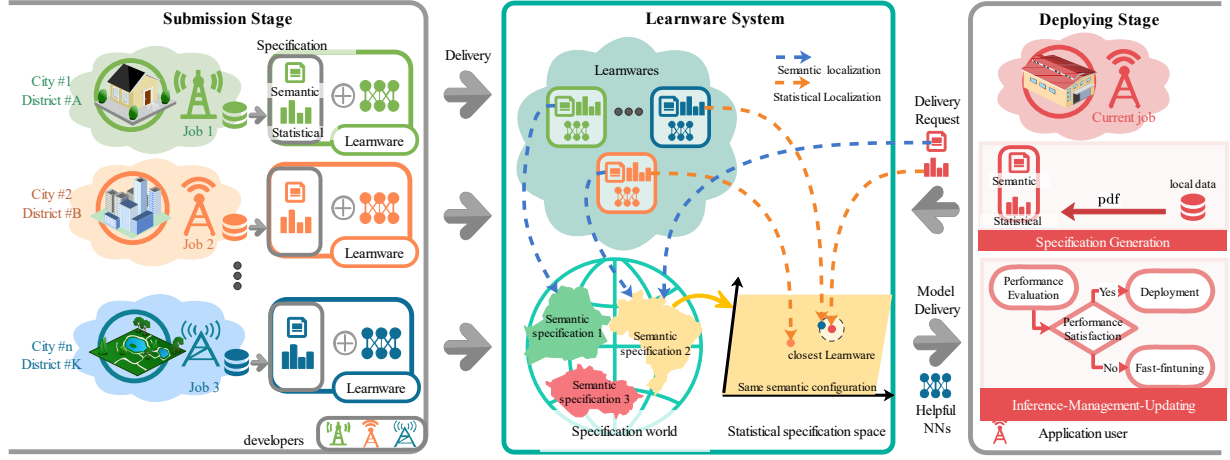


Fig. 4. Learnware paradigm applied to CSI feedback neural networks.

2) *Specification Design*: Due to the diverse functionalities and training data distributions of Learnware models, the specification space is inherently complex and expansive. Without a well-structured design, users may struggle to locate suitable models efficiently. To address this, we introduce the concept of a *specification world*, where functionally similar models are grouped into *specification islands*. Within each island, models differ primarily in their training data distributions, referred to as the data statistical space.

Thus, the overall specification world can be viewed as a coupling of a discrete functional space and a continuous statistical space. For two models with identical functional specifications, a smaller difference in their training data distributions generally implies better generalization performance when transferred to similar environments. As shown in Fig. 4, users first identify the appropriate island based on semantic requirements, and then select the closest model by comparing statistical similarities between their local data and the Learnware training distribution.

Specifications are divided into two categories:

a) *Semantic Specifications*: These consist of string-based descriptors defining model functionality. They help identify the relevant specification island and cover attributes such as task type (e.g., full or implicit feedback, one-side or two-side network), model structure (input/output dimensions, codeword length), and system configurations (bandwidth, frequency, number of carriers, antenna array type). An illustrative example is provided in Tab. III.

b) *Statistical Specifications*: These characterize the distributional properties of the training data and help pinpoint a model's exact position within an island. Due to privacy concerns, the original training data are not shared. Instead, statistical specifications are generated offline at the model developer's side, and packaged with the model and semantic tags before uploading. In our framework, the codeword fingerprint distribution $\mathbf{pdf}$, defined in Eq. 10, is used as the statistical specification. Cosine similarity is employed as the distance metric for retrieval.

3) *Validation of Codeword Fingerprint PDF*: We validate the effectiveness of the codeword fingerprint distribution $\mathbf{pdf}$ as a statistical specification. Following the dataset similarity

evaluation framework established in [41], we investigate the correlation between dataset similarity and model generalization performance. The underlying principle is that if a distance metric meaningfully captures dataset characteristics, then datasets that are close in that metric space should yield similar model performance when models are transferred between them. This validation is essential for our Learnware retrieval mechanism, which relies on $\mathbf{pdf}$ similarity to select appropriate pre-trained models.

To this end, we use 144 scenario-specific simulated datasets (i.e., Learnwares), comprising 72 LOS and 72 NLOS environments. These datasets are arranged counterclockwise according to the azimuth angle relative to a central BS, ensuring full 360-degree coverage for both propagation conditions. Fig. 5(a) presents the Euclidean distance matrix of raw CSI vectors between dataset pairs. Each element (i, j) is calculated as:

$$d_{(i,j)} = \frac{1}{|\mathcal{D}_i| \times |\mathcal{D}_j|} \sum_{\mathbf{v}^{(i)} \in \mathcal{D}_i} \sum_{\mathbf{v}^{(j)} \in \mathcal{D}_j} \|\mathbf{v}^{(i)} - \mathbf{v}^{(j)}\|_2. \quad (11)$$

Fig. 5(b) shows the cosine similarity matrix between the $\mathbf{pdf}$ vectors of the datasets, where each element is given by:

$$\text{sim}_{(i,j)} = \frac{|\mathbf{pdf}_i^H \mathbf{pdf}_j|}{\|\mathbf{pdf}_i\|_2 \|\mathbf{pdf}_j\|_2}. \quad (12)$$

Fig. 5(c) displays the generalization performance matrix. The element $p_{(i,j)}$ indicates the SGCS performance of a model trained on dataset $\mathcal{D}_i$ and tested on $\mathcal{D}_j$. To ensure fairness, all values are normalized by the self-performance $p_{(j,j)}$.

A comparison of the three matrices reveals that Fig. 5(b) aligns more closely with Fig. 5(c) than does Fig. 5(a), suggesting that $\mathbf{pdf}$ is a more effective proxy for predicting model transferability than raw CSI distances. This confirms the suitability of $\mathbf{pdf}$ as a statistical specification for Learnware retrieval. Notably, the cosine similarity matrix in Fig. 5(b) exhibits greater sharpness than the performance matrix in Fig. 5(c). For example, the X-shaped spot in the upper-left corner of Fig. 5(c) is more diffuse than its counterpart in Fig. 5(b), indicating that $\mathbf{pdf}$ similarities serve as a reliable surrogate for functionality proximity. This X-pattern arises

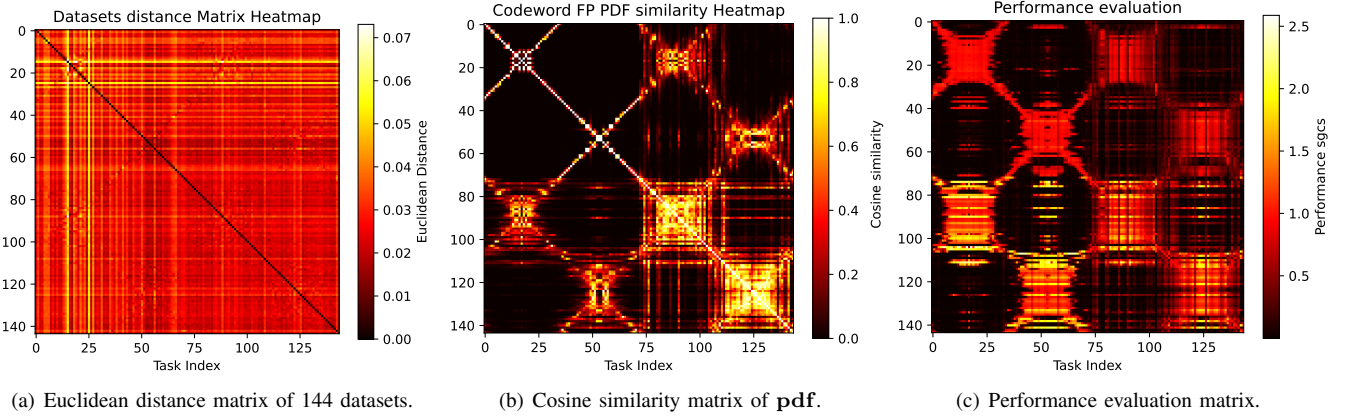


Fig. 5. Comparison between dataset-level distances and model generalization performance.

from the simulation configuration, in which the BS is a north-south ULA and datasets are spatially distributed by angular sectors, yielding a periodic structure in beam directionality.

A closer analysis comparing LOS and NLOS subsets reveals that **pdf** is more sensitive to variations in LOS scenes, as evidenced by the different sharpness of the upper-left submatrix (LOS) and the lower-right submatrix (NLOS) in Fig. 5(b). This is expected since **pdf** in LOS conditions mainly captures the AoD of the direct path, which is sharply defined by the relative geometry between the UE and BS. In contrast, **pdf** in NLOS conditions reflects a broader scattering profile due to multipath propagation, resulting in a more diffuse distribution.

In the early stages of repository development, the diversity of available models may be limited. To mitigate this, initial configuration sets can be used to define semantic specification labels, thereby establishing a range of specification islands. Moreover, simulation-trained CSI models can be employed to populate the repository. As more models are added over time, both semantic and statistical specification coverage will naturally expand.

IV. DEPLOYMENT STRATEGIES

This section describes the deployment stage of our Learnware framework, including the online retrieval mechanism, comparisons with alternative methods, and an optimized search strategy to accelerate model lookup.

A. Deploying Stage

During deployment, the user-side BS first activates Model Management by monitoring for model mismatch. It then submits a model Delivery Request to the Learnware marketplace, which identifies and delivers the most suitable models. This process presents two main challenges: (1) effectively retrieving models that satisfy the user's task requirements; and (2) maximizing the utility of the returned models while minimizing adaptation overhead.

The baseline approach, denoted the Original Model Repository, uploads a subset of local CSI data as a validation set, evaluates every candidate model on this set, and returns the highest-performing model. Although this method achieves high retrieval accuracy, it suffers from several drawbacks:

it compromises data privacy and intellectual property, incurs substantial communication overhead, and scales poorly due to sequential model evaluations. These limitations render it impractical for real-time CSI deployment.

To overcome these issues, our Learnware system employs a specification space retrieval mechanism, as illustrated in Fig. 4. Local CSI data is only used in specification generation, and the specification is sent to the AI data center as the request. The semantic task specifications are used to locate the appropriate specification island. Within that island, statistical specifications are used to index a suitably similar model. Crucially, an exact statistical match is not required. As illustrated in Fig. 5, small discrepancies often result in highly interoperable models, enabling flexible and responsive retrieval. When multiple candidates share identical statistical specifications but differ in network architecture, the BS performs a lightweight on-device evaluation. If the returned model meets the performance threshold, it is deployed directly; otherwise, the BS fine-tunes the model locally.

In the early stages of repository development, coverage of the specification space may be sparse, leading to suboptimal matches. Moreover, the BS may lack knowledge of the ideal local performance, making it difficult to assess the gap between the retrieved model and an optimally trained one. We address this by incorporating a fine-tuning step governed by a user-defined threshold, based on available samples, allowable training iterations, and resource constraints. Models that fail to improve after fine-tuning are discarded in favor of the original, while those that show improvement are adopted for deployment.

B. Post-Retrieval Adaptation

Online deployment of CSI feedback models involves interactions among the AI data center, the BS, and the UE. Initially, the AI data center trains a two-sided neural network and deploys it to the BS for local adaptation. As illustrated in Fig. 6, when the BS detects performance degradation, it issues a model update request [42]. The UE collects new CSI samples and forwards them to the BS, which may either fine-tune its current model, incurring substantial computation and data requirements, or request a new pre-trained model whose

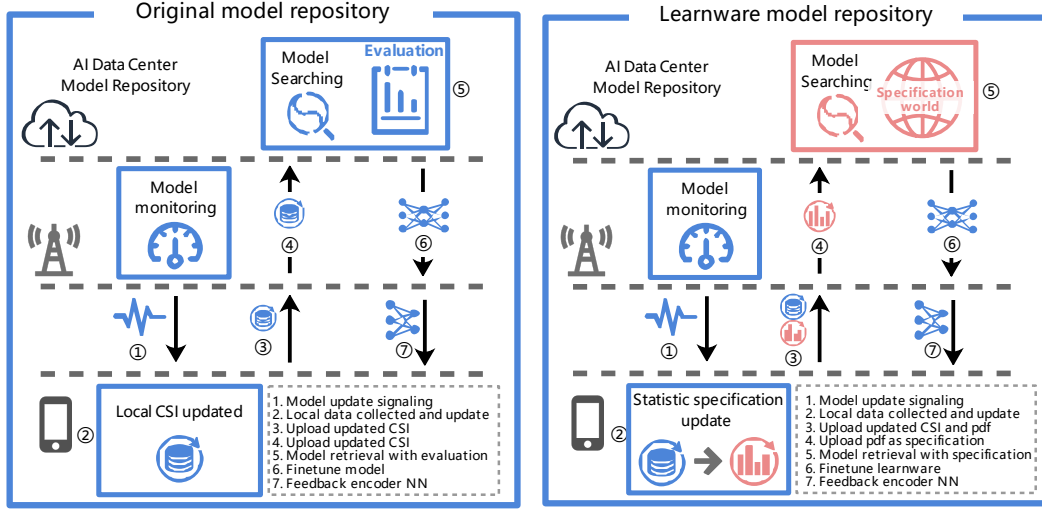


Fig. 6. Online deployment schemes based on model repository, where the key differences are marked in orange.

TABLE II

COMPARISON OF GENERAL MODEL, MODEL SWITCH, AND MODEL REPOSITORY-BASED SCHEMES ACROSS KEY DEPLOYMENT CONSIDERATIONS.

Method	General Model	General Model with fine-tuning	Model Switch	Original Model Repository	Learnware Model Repository
Construction (Initialize)	AI data center BS	Train one model using large-scale data Store a single model	Train several models using large-scale data Store several models	Train many models in parallel Store a single model	
Online deployment	AI data center BS UE	None None None	None Switch model and transmit encoder NN Upload local CSI samples to BS	Model retrieval and return Upload local CSI samples, fine-tune, transmit encoder NN	Model retrieval and return Upload specifications, fine-tune, transmit encoder NN
Merits and Limitations	Accuracy Communication overhead Computation overhead Data privacy	Low Zero Zero ✓	High High High ✓	Middle Low Low ✓	High Middle Middle ✓

training distribution more closely matches the current channel conditions.

As illustrated in Fig.6(a), under the Original Model Repository, the BS must upload raw CSI data for exhaustive candidate evaluation, raising privacy and intellectual property concerns and introducing high latency. In contrast, the Learnware Model Repository requires only compact statistical specifications, reducing raw CSI exposure and significantly reducing retrieval time. Although this approach may slightly reduce selection accuracy, it enables near real-time updates. As illustrated in Fig.6(b), semantic specifications (e.g., antenna configuration and bandwidth) remain fixed during deployment, constraining the search to a single specification island. The BS then submits only the relevant statistical specification for rapid retrieval. For large repositories, the elastic search strategies presented in Section IV-D maintain fast and scalable performance.

C. Comparison with Alternative Methods

We evaluate several baselines against our repository-based solutions:

- **General Model:** A single model trained on a large, diverse dataset and deployed directly to the BS.
- **General Model with fine-tuning:** The general model is fine-tuned at the BS to match local CSI conditions.
- **Model Switch:** Multiple pre-trained models with different configurations; the BS selects the best based on local performance.

- **Original Model Repository:** The BS uploads local data to the AI data center, which evaluates all candidates and returns the best for fine-tuning.
- **Learnware Model Repository:** The BS uploads statistical specifications; the AI data center retrieves the most appropriate pre-trained model based on semantic and statistical matching.

Table II summarizes these approaches across initialization and deployment phases, as well as trade-offs in accuracy, communication cost, computation overhead, and data privacy. During initialization, all methods require substantial training: the General Model and Model Switch focus on fewer, larger models, whereas repository approaches leverage many smaller models for parallelism and scalability. In deployment, General Model and Model Switch require no further AI data center interaction, while repository methods involve continuous communication. The Learnware Model Repository achieves a balanced trade-off, delivering high performance with moderate overhead and preserving user privacy, an equilibrium further validated by our experimental results.

D. Adjustable-Speed Model Search Strategy

To support real-time CSI feedback deployment, we enhance the repository's internal structure to enable rapid and scalable retrieval. After locating the appropriate specification island using semantic attributes, brute-force computation of cosine similarities across all candidates remains impractical for large repositories. As demonstrated in Fig. 5, models

Algorithm 1: Hierarchical LSH Retrieval

Input: Repository $\mathcal{M} = \{\mathbf{p}_i\}_{i=1}^N$ ($\mathbf{p}_i$: normalized spec.);
 query $\mathbf{q}$; levels K ; LSH params $\{(k_\ell, L_\ell)\}_{\ell=1}^K$ (k_ℓ :
 #hash bits, L_ℓ : #tables); stop level k_{stop}
Output: Retrieved anchor model $\mathbf{m}^* \in \mathcal{M}$
 // Offline: build K -level index.
 $\mathbf{W}_\ell \in \mathbb{R}^{L_\ell \times k_\ell \times d}$: random projection matrix;
 1 **for** $\ell = 1, \dots, K$ **do**
 2 Sample $\mathbf{W}_\ell$ with normalized rows;
 3 **for** $\mathbf{p}_i \in \mathcal{M}_\ell, j = 1, \dots, L_\ell$ **do**
 $\mathcal{T}_\ell^{(j)}[h_i^{(j)}] \leftarrow \mathcal{T}_\ell^{(j)}[h_i^{(j)}] \cup \{i\}$ with
 $h_i^{(j)} = \text{PackBits}(\text{sign}(\mathbf{W}_\ell^{(j)} \mathbf{p}_i))$;
 4 **for each bucket** h **with models** $\mathcal{B}$ **do**
 $\mathcal{A}_\ell^{(j)}[h] \leftarrow \arg \max_{\mathbf{p} \in \mathcal{B}} \mathbf{p}^\top \mathbf{c}$: select model closest to
 centroid $\mathbf{c} = \frac{1}{|\mathcal{B}|} \sum_{\mathbf{p}' \in \mathcal{B}} \mathbf{p}'$;
 // Online: cascade.
 $h^{(j)} = \text{PackBits}(\text{sign}(\mathbf{W}_\ell^{(j)} \mathbf{q}))$: query
 signature;
 5 **for** $\ell = 1, \dots, k_{\text{stop}}$ **do**
 6 $\mathbf{m}_\ell \leftarrow \arg \max_{j=1, \dots, L_\ell} \mathbf{q}^\top \mathcal{A}_\ell^{(j)}[h^{(j)}]$; // best
 anchor at level ℓ
 7 **if** $\mathbf{q}^\top \mathbf{m}_\ell > \theta_\ell$ (*confidence threshold*) **then return** $\mathbf{m}_\ell$;
 8 **return** $\mathbf{m}^* \leftarrow \mathbf{m}_{k_{\text{stop}}}$;

with similar statistical specifications tend to be functionally interoperable, offering opportunities to accelerate retrieval. Therefore, we propose an adjustable-speed multi-level LSH strategy (Algorithm 1), which unifies coarse-grained anchor selection and hierarchical indexing into a K -level cascade with early termination capability.

The algorithm consists of two phases. **Offline indexing** builds K levels of LSH tables with increasing hash precision. For each level ℓ , we generate L_ℓ hash tables using k_ℓ -bit signatures ($k_1 < k_2 < \dots < k_K, L_1 > L_2 > \dots > L_K$). This configuration ensures that coarse levels use more tables for high recall to capture a broad candidate set, while finer levels use fewer tables with longer hash keys for precise filtering, balancing retrieval efficiency and accuracy. Each bucket pre-computes an anchor model selected as the one closest to the centroid, enabling $O(1)$ retrieval without enumerating all candidates. **Online retrieval** processes levels sequentially until a specified stopping level k_{stop} , which can be determined by a latency budget τ or confidence thresholds $\{\theta_\ell\}$. At each level, the query specification $\mathbf{q}$ is hashed to retrieve L_ℓ bucket anchors, and the best anchor $\mathbf{m}_\ell$ is selected via cosine similarity. The system can terminate early if $\mathbf{q}^\top \mathbf{m}_\ell > \theta_\ell$, or proceed to finer levels for higher accuracy.

This design provides adjustable speed through two mechanisms: (i) *level selection*: stopping at coarse levels (small k_{stop}) yields sub-millisecond latency suitable for urgent updates, while full cascade (large k_{stop}) ensures near-optimal accuracy; (ii) *confidence-based early exit*: termination triggers when retrieved anchors already satisfy performance requirements, avoiding unnecessary computation. The per-level complexity is $O(L_\ell k_\ell d + L_\ell d)$, independent of repository size N , enabling scalable deployment to billion-model repositories.

TABLE III
SEMANTIC SPECIFICATION LABELS USED IN THE EXPERIMENT.

CSI_Number_Domain	spatial domain
Feedback_Mode	implicit feedback
Structure	two-sided model
Input_Dimension	32×2
Output_Dimension	32×2
Compression_Ratio	1/16
Quantization_Bits	4
BS_Antenna_Array	ULA
Central_Frequency	2.6 GHz
Bandwidth	20 MHz

TABLE IV
BASIC PARAMETER SETTINGS USED IN THE SIMULATION.

Antenna setting	32 ULA antennas at BS with spacing of 0.5 wavelength, centered at (0,0) and arranged in a north-south direction (Y-axis) Single antenna at UE
Center frequency Bandwidth	2.6 GHz 20 MHz
Scene configuration	3GPP-38.901-UMi-LOS 3GPP-38.901-UMi-NLOS
Height	10 m for the BS 1.5 m for the UE
Position of sampling area	Randomly distributed in the 400 m $\times$ 400 m area centered relative to the BS
District range	Circle with a radius of 5 m for Learnware training set Rectangles of various lengths (5 m $\times$ [5,10, 20,30,40,50] m) for target test set
Sampling number per dataset Maps number / Learnware number	10,000 144
CSI Pretreatment	SVD Normalized to real values in the range [0, 1]

V. SIMULATION RESULTS AND DISCUSSIONS

A. Learnware Model Repository Initiation

We initialized the Learnware model library using the implicit feedback CSI two-sided network as an example to construct a specification island. The corresponding semantic labels are shown in Tab. III.

1) *CSI Simulation*: Before collecting models, we generated simulation datasets using the configurations listed above to train Learnware models and enrich the specification island. We created 144 Learnwares using QuaDRiGa [43] under the 3GPP-38.901-UMi configuration (72 LOS + 72 NLOS, each with 10,000 samples). CSI sampling covered full 360° azimuth diversity with independent environment layouts per dataset (i.e., the environment layout was restarted for each dataset). Details are in Tab. IV and Fig. 2(a). The angular distribution characteristics are reflected in the pdf shown in Fig. 2(b).

2) *NN Training and Evaluation*: Each Learnware was trained using a lightweight CNN-based autoencoder (CsiNet [8]) with 2D convolutions replaced by 1D, kernel size expanded from 3 to 7). The General Model follows the transformer-based EVCsiNet-T [44] (details in Tab. V), aligning with the 3GPP R18 AI/ML study [44], [45]. The dataset was partitioned into training (85%), validation (10%), and test (5%) sets. We used the Adam optimizer with a learning rate of 1e-4, training for 1,000 epochs and saving the model at the point of optimal validation performance.

For targeted testing, QuaDRiGa generated CSI data matching the semantic configurations. Test set sampling areas were rectangles of various lengths (5 m $\times$ [5, 10, 20, 30, 40, 50] m) with random centers and rotations under the 3GPP-38.901-UMi LOS/NLOS configuration. Our solution retrieves the best Learnware based on statistical specification similarity

TABLE V
AUTOENCODER STRUCTURES FOR CNN AND TRANSFORMER-BASED CSI
FEEDBACK

Layer	Type	Input	Output	Details
CNN-based Autoencoder [8]				
Input	-	(32, 2)	(32, 2)	Real and imaginary parts
Encoder	Conv1D	(32, 2)	(32, 2)	Kernel: 7×1 , BN, ReLU
	Reshape	(32, 2)	(64,)	Flatten
	Dense+Sigmoid	(64,)	(4,)	Compression ratio $CR = 1/16$
	Quantization	(4,)	(16,)	$Q = 4$ bits, feedback_bits=16
Decoder	Dequantization	(16,)	(4,)	-
	Dense	(4,)	(64,)	-
	Reshape	(64,)	(32, 2)	-
	RefineNet Block $\times 2$	(32, 2)	(32, 2)	Conv1D: (32, 2) $\rightarrow$ (32, 8) Conv1D: (32, 8) $\rightarrow$ (32, 16) Conv1D: (32, 16) $\rightarrow$ (32, 2) Skip Connection
	Conv1D	(32, 2)	(32, 2)	Kernel: 7×1 , BN, ReLU
	Sigmoid	(32, 2)	(32, 2)	Output activation
Transformer-based Autoencoder [44]				
Input	Input	(32, 2)	(32, 2)	$N_{\text{Token}} = 32$, $feature_dim=2$
	Linear Embedding	(32, 2)	(32, 81)	$Embedding_dim = 81$, Dense
	Positional Encoding	(32, 81)	(32, 81)	Sinusoidal encoding
Encoder	Transformer Block $\times L_{TF} (L_{TF} = 12)$	(32, 81)	(32, 81)	Multi-Head Attn: $N_{\text{head}} = 3$, Dropout(0.1) LayerNorm, Residual connection
	LayerNorm	(32, 81)	(32, 81)	FFN: $324 \rightarrow 81 (4 \times)$, ReLU, Dropout(0.1)
	Dense	(32, 81)	(32, 2)	LayerNorm, Residual connection
	Flatten	(32, 2)	(64,)	Final normalization
	Dense+Sigmoid	(64,)	(4,)	Project back to 2 channels
	Quantization	(4,)	(16,)	$CR = 1/16$ $Q = 4$ bits, feedback_bits=16
Decoder	Dequantization	(16,)	(4,)	-
	Dense	(4,)	(64,)	-
	Reshape	(64,)	(32, 2)	-
	Linear Embedding	(32, 2)	(32, 81)	Dense(81), scale by $\sqrt{81}$
	Positional Encoding	(32, 81)	(32, 81)	Sinusoidal encoding
	Transformer Block $\times L_{TF} (L_{TF} = 12)$	(32, 81)	(32, 81)	Multi-Head Attn: $N_{\text{head}} = 3$, Dropout(0.1) LayerNorm, Residual connection
	LayerNorm	(32, 81)	(32, 81)	FFN: $324 \rightarrow 81 (4 \times)$, ReLU, Dropout(0.1)
	Dense	(32, 81)	(32, 2)	LayerNorm, Residual connection Final normalization Output projection

and returns it for optional fine-tuning. We evaluated both the initial and fine-tuned performance.

B. Performance Gain

Performance evaluations are conducted for the aforementioned schemes, all sharing the same architectural design but differing in their training processes:

- **General Model (with/without fine-tuning):** A transformer-based (architecture in Tab. V) model with high generalization capability is trained using a mixture of the 144 datasets from the training Learnwares.
- **Model Switch:** Four configuration-specific models are prepared with CNN-based architecture in Tab. V. The 144 datasets are divided into four groups based on LOS/NLOS conditions and the upper/lower sides of the ULA axis at the BS (north/south sides in Fig. 2(a), corresponding to the first/second halves in Fig. 2(b)). Each model is trained with a mix of 36 datasets from each group.
- **Original Model Repository:** The performance of the CNN-based 144 models in the repository is evaluated to select the best-performing model, which represents the upper bound of performance for model repository-based schemes.
- **Learnware Model Repository:** The codebook fingerprint sample frequency **pdf** is adopted as the statistical specification, with N set to 64. The model library computes the cosine similarity of **pdf** between all Learnwares within the specified island¹ and the target scene test set, selecting the Learnware with the highest similarity.

¹Given the limited number of Learnwares in this experiment, techniques such as LSH will be implemented to accelerate retrieval when scaling up.

- **Upper Bound:** With sufficient local data, smaller models often achieve better overfitting performance than generalized models, thus setting the performance upper bound. Due to the large sampling area in the test scenario, datasets ranging from 10,000 to 50,000 samples are used for training to obtain this upper bound.

1) *Initial Performance Verification:* We first evaluate performance without fine-tuning. Tab. VI first validates the strong performance of the scene-specific model compared to General Model, where the SGCS performance is the mean of the 10 cases included in each sampling area range. Specific CNN models (2k parameters, 0.11M FLOPs) are compared against transformer-based General Models with increasing complexity (TF-1 to TF-4, with millions of parameters). Results reveal that specific models consistently outperform all General Models across all scenarios, even when General Model capacity expands to $1,000\times$ the parameter count and $5,600\times$ the computational cost. This demonstrates that specific advantages persist over an extremely wide capacity range, with million-parameter models still failing to match small specialized models. The diminishing returns in General Model performance also highlight the inefficiency of simply scaling up model capacity. Moreover, deploying such large models imposes significant hardware demands and inference latency, making them impractical for resource-constrained edge devices, whereas lightweight scene-specific models enable efficient on-device deployment.

Tab. VII analyzes the sensitivity of all methods to different compression ratios and quantization bits. The Learnware Model Repository consistently outperforms the General Model across all configurations, with SGCS gains of 0.05–0.45 (6.4%–128.2%) in LOS and 0.05–0.09 (12.4%–60.4%) in NLOS. Even under the most constrained feedback ($CR=1/32$, $Q = 4$), the Learnware achieves 0.794 (LOS) and 0.215 (NLOS), far exceeding the General Model (0.348 LOS, 0.134 NLOS), demonstrating robustness to feedback payload variations.

Fig. 7 displays representative individual cases randomly selected from the 10 LOS and 10 NLOS test scenarios with $5m \times 5m$ sampling area. In each subfigure, the blue bars represent the SGCS performance of all 144 LOS Learnware models when tested on that specific LOS/NLOS test scenario, while orange bars represent NLOS Learnware models. The green-highlighted bar indicates the Learnware model selected by our codebook fingerprint method (i.e., the model whose training dataset's **pdf** has the highest cosine similarity with the test scenario's **pdf**). The horizontal dashed lines show the performance of five methods evaluated on the test scenario. The performance ranking consistently follows: Upper Bound > Original Model Repository > Learnware Model Repository > General Model/Model Switch. In both scenarios, repository-based selections closely approach the Upper Bound, significantly outperforming General Model by approximately 18.8% (0.147/0.781) in LOS and 57.7% (0.169/0.293) in NLOS, demonstrating the effectiveness of **pdf**-based model selection across different propagation conditions. Extensive testing reveals that Learnware selection is robust to LOS/NLOS conditions: green bars (selected models) predominantly appear

TABLE VI
AVERAGE SGCS PERFORMANCE OF SCENE-SPECIFIC SMALL MODELS AND LARGE GENERAL MODEL WITH VARIOUS COMPLEXITY. CR=1/16 AND FEEDBACK QUANTIZATION BITS $Q=4$.

Sample Range (m^2)	LOS								NLOS							
	Scene-specific		Learnw. Model Repository	General Model (various complexity) ¹					Scene-specific		Learnw. Model Repository	General Model (various complexity)				
	CNN ²	TF-1		CNN	TF-1	TF-2	TF-3	TF-4 ³	CNN	TF-1		CNN	TF-1	TF-2	TF-3	TF-4
5 × 5	0.915	0.915	0.889	0.332	0.628	0.725	0.755	0.777	0.350	0.357	0.317	0.181	0.208	0.207	0.214	0.230
5 × 10	0.881	0.883	0.855	0.340	0.701	0.719	0.746	0.715	0.347	0.343	0.321	0.225	0.213	0.220	0.245	0.263
5 × 20	0.879	0.886	0.808	0.297	0.703	0.710	0.713	0.715	0.330	0.347	0.314	0.162	0.205	0.205	0.216	0.221
5 × 30	0.868	0.881	0.721	0.292	0.651	0.684	0.691	0.660	0.324	0.339	0.303	0.187	0.213	0.207	0.214	0.213
5 × 40	0.848	0.858	0.703	0.290	0.521	0.579	0.598	0.613	0.334	0.350	0.285	0.189	0.217	0.219	0.230	0.238
5 × 50	0.844	0.865	0.652	0.284	0.489	0.491	0.467	0.492	0.276	0.286	0.232	0.126	0.189	0.188	0.188	0.194
Parameters [10 ³]	2.01	58.66	2.01	2.01	58.66	424.29	1,069.56	2,114.99	2.01	58.66	2.01	2.01	58.66	424.29	1,069.56	2,114.99
Compare with CNN	1×	29×	1×	1×	29×	211×	532×	1,051×	1×	29×	1×	1×	29×	211×	532×	1,051×
FLOPs [10 ⁶]	0.11	12.64	0.11	0.11	12.64	104.03	312.06	624.09	0.11	12.64	0.11	0.11	12.64	104.03	312.06	624.09
Compare with CNN	1×	115×	1×	1×	115×	945×	2,837×	5,673×	1×	115×	1×	1×	115×	945×	2,837×	5,673×

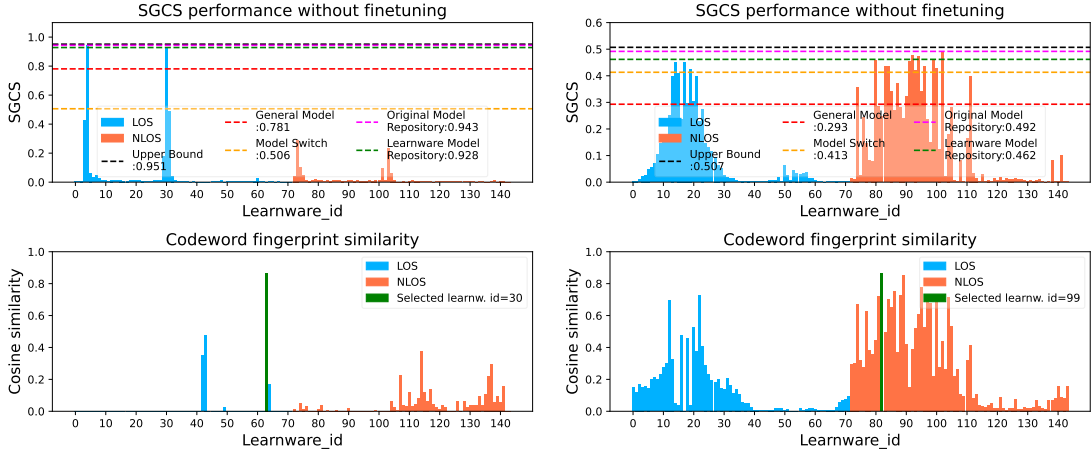
¹ Architectures in Table V: CNN (same with Scene-specific and Learnware Model Repository); TF-1: Transformer with $L_{TF} = 2$ and $Embedding_dim = 27$; TF-2: Transformer with $L_{TF} = 2$ and $Embedding_dim = 81$; TF-3: Transformer with $L_{TF} = 6$ and $Embedding_dim = 81$; TF-4: Transformer with $L_{TF} = 12$ and $Embedding_dim = 81$.

² The bold-highlighted CNN is adopted for scene-specific models as the Upper Bound.

³ The bold-highlighted TF-4 is adopted as the default NN architecture for the General Model in subsequent experiments.

TABLE VII
AVERAGE SGCS PERFORMANCE OF METHODS WITH VARIOUS CR AND FEEDBACK QUANTIZATION BITS. SAMPLING RANGE: $5 \times 5 m^2$.

Configurations		LOS scenes					NLOS scenes				
CR	Quantization bits (Q)	General Model	Model Switch	Learnware Model Repository	Original Model Repository	Upper Bound	General Model	Model Switch	Learnware Model Repository	Original Model Repository	Upper Bound
1/16	2	0.681	0.371	0.832	0.841	0.878	0.219	0.197	0.296	0.309	0.336
	3	0.715	0.453	0.850	0.859	0.881	0.230	0.222	0.313	0.328	0.356
	4	0.777	0.477	0.889	0.891	0.915	0.230	0.230	0.317	0.333	0.350
	5	0.781	0.482	0.899	0.907	0.921	0.232	0.233	0.320	0.335	0.356
1/32		0.348	0.249	0.794	0.816	0.852	0.134	0.144	0.215	0.239	0.251
1/16	4	0.777	0.477	0.889	0.891	0.915	0.230	0.230	0.317	0.333	0.350
1/8		0.843	0.681	0.897	0.906	0.920	0.420	0.385	0.472	0.489	0.510



(a) Testing LOS scene.

(b) Testing NLOS scene.

Fig. 7. SGCS performance without fine-tuning and codeword fingerprint cosine similarity. The LOS and NLOS target testing scenes are presented in (a) and (b). In each subfigure, the LOS and NLOS Learnwares are displayed in blue and orange, respectively. The Learnware selected by the codebook fingerprint (i.e., the one with the maximum cosine similarity) is marked in green. The CSI sampling area for the test set is set to $5 m \times 5 m$, the same as the Learnware training set. CR=1/16 and feedback quantization bits $Q=4$.

among blue bars (LOS Learnwares) for LOS test scenarios, and among orange bars (NLOS Learnwares) for NLOS scenarios. Hence, LOS/NLOS is excluded from semantic specifications.

As shown in Fig. 7, it is hard to decide from a predefined threshold whether a Learnware requires fine-tuning, because the Upper Bound is relatively high for LOS test scenarios but relatively low for NLOS scenarios, making fine-tuning necessary in the latter. To improve Learnware performance in NLOS scenarios, one approach is to increase the feedback overhead by switching to a different specification island.

TABLE VIII
SGCS PERFORMANCE BEFORE AND AFTER FINE-TUNING (500 SAMPLES, 100 EPOCHS) OF INDIVIDUAL REPRESENTATIVE CASE, CORRESPONDING TO FIG. 8.

Methods	LOS Test Scene		NLOS Test Scene	
	Without fine-tuning	With fine-tuning	Without fine-tuning	With fine-tuning
General Model	0.781	0.936	0.293	0.408
Model Switch	0.506	None	0.413	None
Original Model Repository	0.943	0.951	0.492	0.499
Learnware Model Repository	0.928	0.946	0.462	0.472
Upper Bound	0.951		0.507	

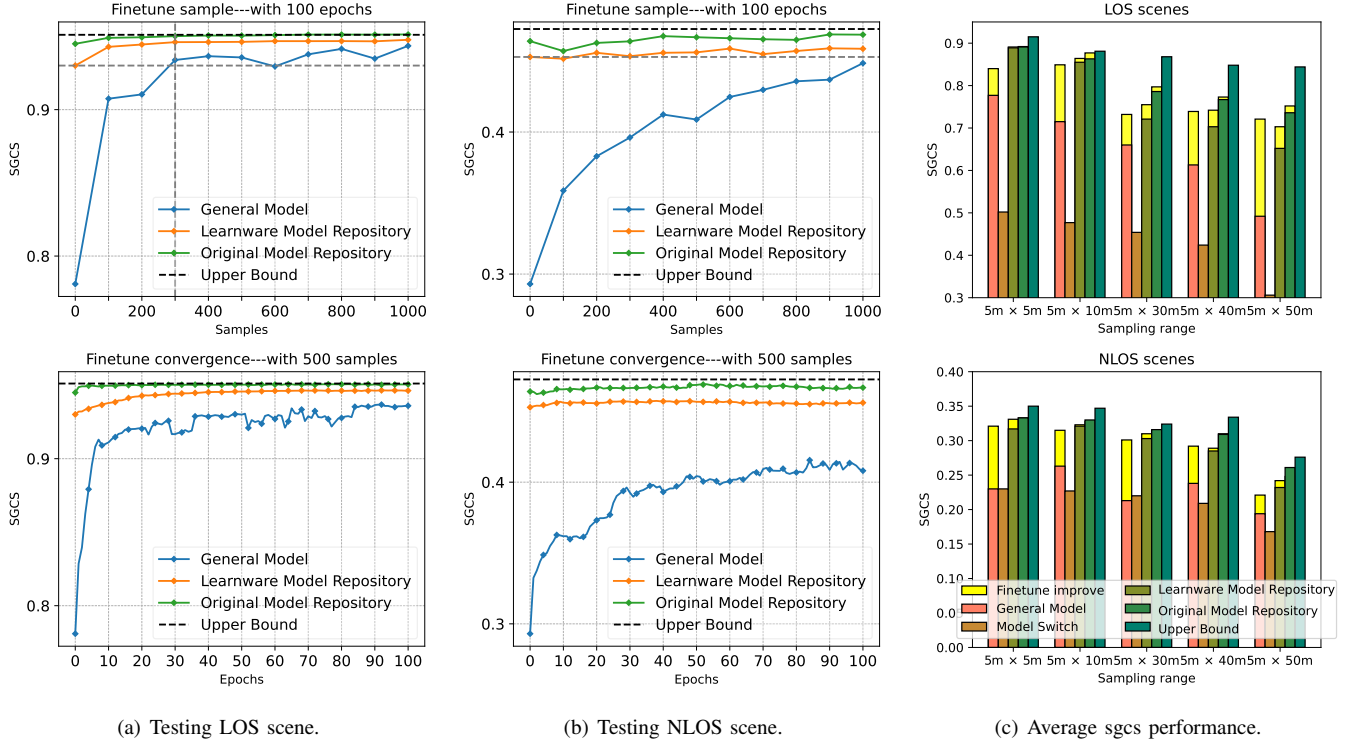


Fig. 8. Performance improvements on the two cases from Fig. 7 post-fine-tuning. The LOS and NLOS target test scenarios are shown in (a) and (b), respectively. In each subfigure, the upper and lower sections represent the number of samples used for fine-tuning and the number of fine-tuning epochs, respectively. Tested CSI sampling area: $5\text{ m} \times 5\text{ m}$. The corresponding fine-tuning results are summarized in Tab. VIII. (c) presents the average SGCS performance over multiple sampling areas, where each bar represents the average across 10 test scenarios. Fine-tuning overhead: 500 samples and 100 epochs.

2) *Fine-Tuning Performance*: The convergence and sample requirements for fine-tuning are depicted in Figs. 8(a) and 8(b) (same cases from Fig. 7), with detailed SGCS performance before and after fine-tuning provided in Tab. VIII. The results demonstrate that minimal fine-tuning of the selected Learnware can effectively boost performance, achieving parity with the Original model repository and, in some cases, even match the Upper bound. Learnware fine-tuning overhead is minimal, as it provides a better initialization point than General Model. For example, in the LOS Finetune-sample scenario, General Model requires 300 samples to match the initial performance of the Learnware after 100 fine-tuning epochs (indicated by the gray dashed lines), meaning that direct Learnware deployment saves at least 300 samples times 100 epochs of fine-tuning effort compared to General Model. In the NLOS scenario, the General Model needs up to 1000 samples over 100 epochs to catch up with the Learnware's initial performance. This demonstrates that direct Learnware deployment saves 300 to 1000 training samples (depending on propagation conditions) compared to fine-tuning the General Model from scratch, while achieving superior final performance after fine-tuning.

We also conducted experiments on multiple test scenarios across different sampling areas, each consisting of 10 LOS and 10 NLOS cases. The averaged performance is presented in Fig. 8(c). The fine-tuning overhead was set to 500 samples for 100 epochs, with yellow bars indicating the corresponding fine-tuning gain. The results confirm that repository-based approaches provide superior initialization, and in some cases, outperform General Model even after its fine-tuning. Addi-

tionally, the slower performance improvement observed for Learnware compared to General Model is due to its weaker generalization ability and its closer proximity to the upper bound. This proximity makes it more challenging to reach the optimal solution and slows down the gradient descent process.

C. Adjustable-Speed LSH

We evaluate the proposed adjustable-speed multi-level LSH retrieval strategy, validating the accuracy-latency trade-offs and demonstrate the practical feasibility for real-time CSI feedback deployment. We create a large-scale specification space containing 100 million statistical specification vectors. First, we generate 1,000 CSI datasets using QuaDRiGa simulations under diverse urban microcell scenarios and compute their **pdf**. We then fit a Gaussian Mixture Model (GMM) with 50 components to these real **pdf** vectors and perform data augmentation by sampling 100 million normalized 64-dimensional specification vectors. The repository is organized into four LSH levels, with various L hash tables and k -bit signatures, yielding next-level's candidate set sizes of 6.25M, 195k, 3k, and 48 models. We test 1,000 query specifications (from the same GMM distribution) and measure average cosine similarity between retrieved anchors and queries (i.e., LSH score). The brute-force upper bound is computed similarly by exhaustive search over the entire repository (100 million vectors). Relative Accuracy is the ratio of LSH Score to Brute-force Upper Bound. Speedup Factor compares the cumulative LSH retrieval time against brute-force search (5,000 ms for 100 million vectors).

TABLE IX
PERFORMANCE OF MULTI-LEVEL LSH WITH ADJUSTABLE SEARCH DEPTH

Level	k	L	Input Candidate Repository Size	LSH Score	Brute-force Upper Bound	Relative Accuracy	Per-Level Time (ms)	Cumulative Time (ms)	Speedup Factor
L1	4	25	1.0×10^8	0.85	0.99	85.9%	0.25	0.25	$20,000\times$
L2	5	18	6.25×10^6	0.91	0.99	91.9%	0.35	0.60	$8,333\times$
L3	6	12	1.95×10^5	0.96	0.99	97.0%	0.30	0.90	$5,556\times$
L4	7	6	3.05×10^3	0.98	0.99	99.0%	0.20	1.10	$4,545\times$
Final	-	-	48	0.99	0.99	100%	0.05	1.15	$4,348\times$

TABLE X
ONLINE DEPLOYMENT COST COMPARISON FOR THE GENERAL MODEL, MODEL SWITCH, AND PROPOSED MODEL REPOSITORY-BASED SCHEMES.

Deployment considerations	Communication overhead		Time complexity		Communication overhead [KB] ¹		Time complexity [s] ²	
	Data center-BS	BS-UE	Fine-tuning	Model retrieval	Data center-BS	BS-UE	Fine-tuning	Model retrieval
General Model	None	None	None	None	0	0	0	0
General Model + fine-tuning	None	500 samples for fine-tuning; return 1 encoder	100 epochs, 500 samples	None	0	$240 + 3.37 \times 10^4$	373.5	0
Model Switch	None	500 samples for evaluation; return 1 encoder	None	Evaluate 4 models	0	$240 + 5.8$	0	0.71
Original model repository	500 samples for evaluation; return 1 model	500 samples for fine-tuning; return 1 encoder	100 epochs, 500 samples	Evaluate 144 models	$240 + 31.7$	$240 + 5.8$	61.6	25.7
Learnware model repository	Specification; return 1 model	500 samples for fine-tuning; return 1 encoder	100 epochs, 500 samples	Retrieve from 144 models	$2.6 + 31.7$	$240 + 5.8$	61.6	$1.48e-5$

¹ File sizes: 500 samples = 240 KB; CNN autoencoder = 31.7 KB; CNN encoder = 5.83 KB; TF-4 autoencoder = 67.5 MB; TF-4 encoder = 33.7 MB; semantic specification = 0.25 KB; statistical specification = 2.36 KB.

² All measurements are obtained using a Tesla V100 GPU. Fine-tuning 500 samples for 100 epochs averaged 373.5 seconds for TF-4 Transformer and 61.6 for CNN. Model evaluation took 0.178 seconds per model. Cosine similarity computation for all 144 Learnwares averaged $1.48e-5$ seconds.

Tab. IX summarizes the performance of our multi-level LSH strategy across different search depths. Deeper search levels yield higher accuracy but incur greater cumulative latency, enabling users to select the appropriate trade-off based on their requirements. Shallower levels (L1-L2) provide ultra-fast retrieval (0.25–0.60 ms) with good accuracy (85.9–91.9%), while deeper levels (L3-L4) deliver near-optimal accuracy (97.0–99.0%) at moderately higher latencies (0.90–1.15 ms). All cumulative latencies (0.25–1.15 ms) remain well within the typical CSI feedback cycles of 10–80 ms in 5G NR systems [46], confirming the practical feasibility of our approach.

D. Complexity Analysis

In Tab. X, we quantitatively compare the online deployment costs of General Model, Model Switch, and the proposed model repository-based schemes, including communication overhead and time complexity. Given a fine-tuning budget of 500 samples over 100 epochs, and with model performance evaluation also requiring 500 samples, communication overhead is categorized into two segments: from the AI data center to the BS, and from the BS to the UE. Only the model repository-based schemes incur communication costs in the former.

The Learnware system addresses the Original model repository's drawback of transmitting local data by significantly reducing transmission overhead from 240 KB to 2.6 KB. Notably, the TF-4 General Model exhibits substantially higher communication costs: its encoder size reaches 33.7 MB, three orders of magnitude larger than the CNN encoder (5.83 KB), making frequent model updates impractical for bandwidth-constrained scenarios.

Time complexity is critical for online deployment and is divided into fine-tuning time and model retrieval time. Fine-tuning the TF-4 Transformer requires 373.5 seconds, over six times longer than the CNN-based model (61.6 seconds), due to its massive parameter count (2.1 million). This becomes a significant disadvantage for the General Model + fine-tuning

scheme, as the system must fall back to traditional feedback during this extended period, potentially degrading feedback quality. In contrast, the Learnware model repository requires only $1.48e-5$ seconds for retrieval, which corresponds to the cost of computing inner products and is thus negligible. Even when scaled to billion-model repositories, our adjustable-speed multi-level LSH (Algorithm 1) maintains retrieval latency within 0.25–1.15 ms (Tab. IX), well below the 10–80 ms CSI feedback cycles specified in 3GPP standards [46]. The near-instantaneous response ensures that during fine-tuning, the Learnware model can be deployed immediately while the locally fine-tuned model trains in the background. This scalability, combined with the minimal communication overhead of statistical specifications, positions the Learnware framework as a practical solution for real-world deployments where both bandwidth and latency are constrained.

E. Further Discussion

1) *Validation for pdf Selection:* To validate the accuracy of codebook fingerprint pdf selection, we analyzed both the ranking of the Learnware selected by pdf and its performance gap compared to the top-performing model, as illustrated in Fig. 9. We simulated 1,000 LOS and 1,000 NLOS test scenarios for each sampling area, with each scenario containing 500 samples (used to generate pdf and to evaluate performance). This extensive simulation ensures robust probability estimations.

Fig. 9(a) shows the probability that the test performance of the pdf-selected Learnware ranks among the top x positions in the model repository, which consists of 144 models. In over 90% of cases, the ranking falls within positions 25 to 27. Additionally, the probability of ranking in the top three is 45% to 55% for LOS scenarios and 25% to 35% for NLOS scenarios.

A more intuitive indicator is the SGCS performance gap between the pdf-selected Learnware and the top-performing model ($SGCS_{OMR} - SGCS_{LMR}$). Fig. 9(b) presents a boxplot

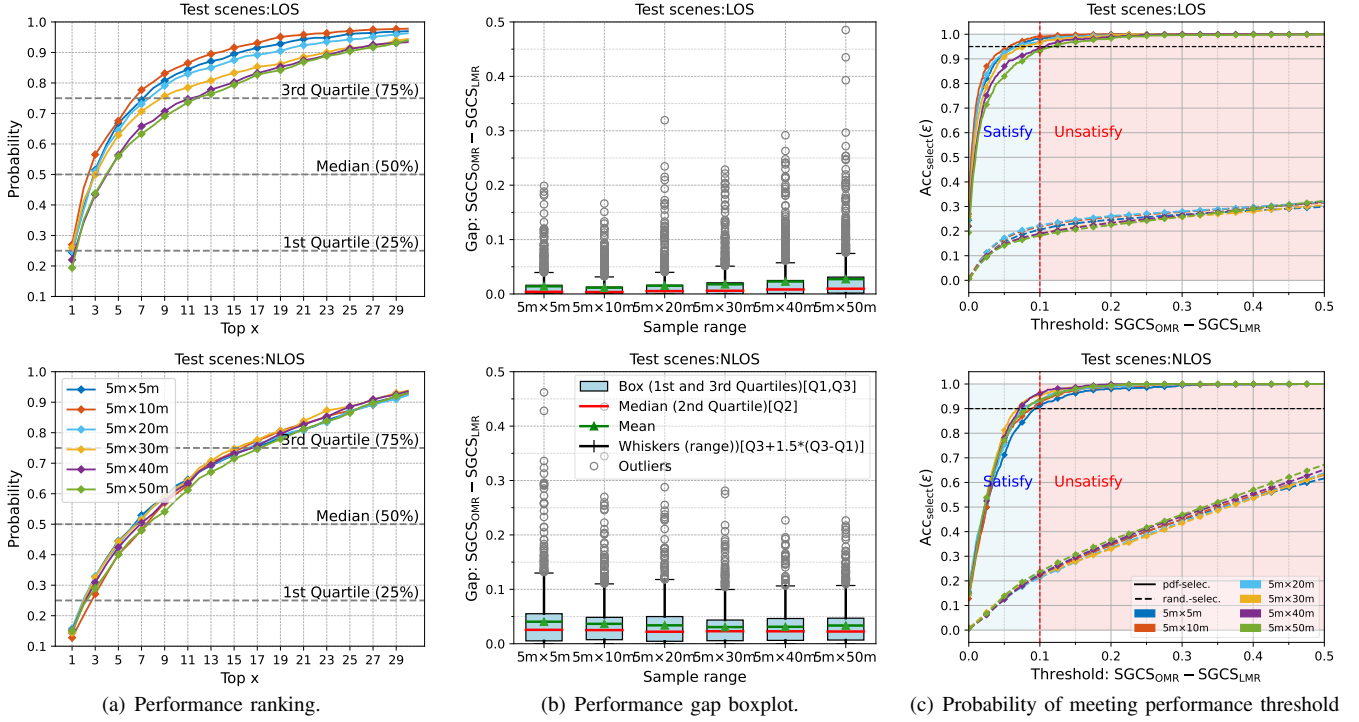


Fig. 9. We simulated 1,000 LOS and 1,000 NLOS scenarios in each sampling area, with each scenario comprising 500 samples. (a) Performance ranking of the learnware selected by **pdf** within the model repository (comprising 144 models), i.e., the probability of being ranked in the top x positions; (b) box-plot of the performance gap between the **pdf**-selected learnware and the top-performing model (i.e., Original model repository); (c) Selection accuracy: probability that the performance gap of the **pdf**-selected learnware meets the expected threshold.

of the performance gap distribution. The lower boundary, median line, and upper boundary of each box represent the first quartile (Q1), median (Q2), and third quartile (Q3), respectively. The box length reflects the interquartile range of the data. In LOS scenarios, the shorter boxes indicate a more concentrated performance gap distribution, which becomes more dispersed as the sampling area increases. Conversely, in NLOS scenarios, the distribution becomes more concentrated with larger sampling areas, since the performance degradation of the top-performing model offsets the impact of less accurate **pdf** selection. The median line is consistently biased towards the lower end of the box, indicating that the performance gap is generally small. The whiskers represent the range of the data and help identify potential outliers. The results show that even in outlier cases, the performance gap does not exceed 0.5.

To quantitatively assess selection accuracy, we define it as the probability that the performance gap falls within a threshold ϵ :

$$\text{Acc}_{\text{select}}(\epsilon) = \Pr(|\text{SGCS}_{\text{OMR}} - \text{SGCS}_{\text{LMR}}| \leq \epsilon), \quad (13)$$

Fig. 9(c) presents selection accuracy with $\epsilon = 0.1$. The light blue area indicates acceptable cases ($\text{gap} \leq 0.1$), while the light red area indicates failure ($\text{gap} > 0.1$). The results demonstrate approximately 95% accuracy for LOS and 90% for NLOS scenarios, significantly exceeding random selection.

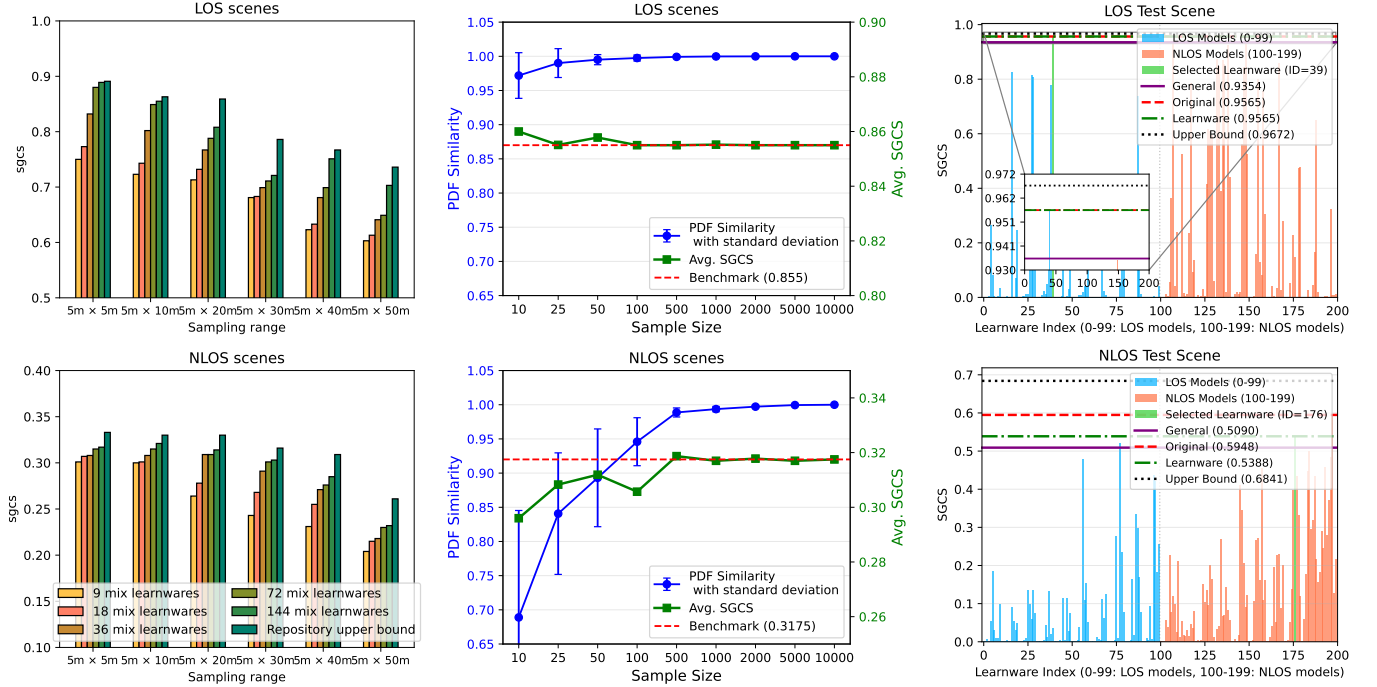
2) Impact of Learnware Quantity: We further investigate the impact of the Learnware model library size on performance. As shown in Fig. 10(a), we randomly selected varying numbers of Learnwares from a pool of 144 to form

model libraries of different sizes. This process was repeated 100 times to reduce the effects of randomness. The mean performance across 10 test scenarios was then computed for each sampling region. The last column in each group of bars represents the performance of the best Learnware among the 144, corresponding to the Original model repository.

The experimental results indicate that as the model library size increases, the performance of the selected Learnwares also improves. However, this improvement exhibits diminishing marginal returns, suggesting that the model library size should be kept within a reasonable range to balance performance with retrieval efficiency. Additionally, as the sampling region of the test scenarios expands, the performance gap between the Original model repository and the selected Learnwares increases, particularly in NLOS scenarios. This is attributed to the more dispersed angular distribution of the datasets, which reduces the effectiveness of **pdf** as a statistical specification criterion. Therefore, when deploying the system, the BS should carefully define local scene boundaries to avoid excessively dispersed angular distributions.

3) Impact of Sample Size on Model Selection Performance: We investigate how the number of CSI samples affects **pdf** estimation accuracy and subsequent model selection performance. In practical deployment, BSs may have limited samples for statistical specification generation, making this analysis essential for understanding real-world applicability.

Fig. 10(b) illustrates the impact of sample size on both PDF similarity and SGCS performance. The blue curves with error bars represent PDF similarity, the green curves show average SGCS, and the red dashed lines indicate the benchmark SGCS



(a) Learnware model repository performance with various numbers of learnwares. (b) Sample size impacts for pdf and SGCS in Learnware Model Repository. (c) SGCS performance without fine-tuning in OFDM scenario.

Fig. 10. (a) Average SGCS performance of Learnware Model repository, with each group of 10 cases. CR=1/16 and feedback quantization bits $Q=4$; (b) Test scenarios: 10 cases for LOS/NLOS, $5m \times 5m$ sampling range. For each sample size, the estimated pdf is compared against the benchmark (full 10,000 samples). Performance loss is defined as $SGCS_{\text{benchmark}} - SGCS_{\text{other}}$. PDF similarity measures the cosine similarity between the estimated pdf and the benchmark pdf. (c) Multi-carrier OFDM scenario without fine-tuning. The configuration and legend follow the same format as Fig. 7.

TABLE XI
SIMILARITY AND DISTANCE METRICS FOR PROBABILITY DISTRIBUTIONS

Metric	Original Formula	Original Range	Normalized Similarity	Complexity ¹
Cosine Similarity	$s_{\text{cos}}(p, q) = \frac{\sum_i p_i q_i}{\ p\ \cdot \ q\ }$	[0, 1]	$S_{\text{cos}} = s_{\text{cos}}$	Low
Bhattacharyya Coefficient [47]	$s_{\text{Bhatt}}(p, q) = \sum_i \sqrt{p_i q_i}$	[0, 1]	$S_{\text{Bhatt}} = s_{\text{Bhatt}}$	Medium
Hellinger Distance [48]	$d_{\text{Hell}}(p, q) = \frac{1}{\sqrt{2}} \ \sqrt{p} - \sqrt{q}\ _2$	[0, 1]	$S_{\text{Hell}} = 1 - d_{\text{Hell}}$	Medium
Total Variation	$d_{\text{TV}}(p, q) = \frac{1}{2} \sum_i p_i - q_i $	[0, 1]	$S_{\text{TV}} = 1 - d_{\text{TV}}$	Low
Jensen-Shannon Divergence [49]	$d_{\text{JS}}(p, q) = \frac{1}{2} D_{\text{KL}}(p \ m) + \frac{1}{2} D_{\text{KL}}(q \ m)$	[0, 1]	$S_{\text{JS}} = 1 - d_{\text{JS}}$	High
KL Divergence [50]	$d_{\text{KL}}(p, q) = \sum_i p_i \log \frac{p_i}{q_i}$	$[0, +\infty)$	$S_{\text{KL}} = \exp(-d_{\text{KL}})$	High
Euclidean Distance	$d_{\text{Euc}}(p, q) = \sqrt{\sum_i (p_i - q_i)^2}$	$[0, \sqrt{2}]$	$S_{\text{Euc}} = 1 - \frac{d_{\text{Euc}}}{\sqrt{2}}$	Low

¹ Complexity levels: Low (basic arithmetic operations), Medium (element-wise square roots), High (element-wise logarithmic or exponential operations).

performance. The results demonstrate that the Learnware Model Repository model exhibits strong robustness to sample size variations, with performance degradation negligible across all tested scenarios. For LOS scenes, characterized by concentrated angular energy, even 25 samples yield near-perfect PDF similarity (> 0.99), and the SGCS performance stabilizes at the benchmark level, confirming that sample size has minimal impact on model selection in such environments. For NLOS scenes, while PDF estimation quality improves progressively with sample size due to diffuse multipath scattering, the impact on SGCS performance remains remarkably small. Remarkably, with only 50 samples, the SGCS already exceeds 98% of the benchmark, and with 500 samples, the performance gap be-

comes virtually indistinguishable. These findings confirm that BSs can collect as few as 500 samples to obtain reliable model selection performance regardless of propagation conditions, eliminating the need for extensive data collection in practical deployments.

4) *Comparison of Distributional Similarity Metrics:* Given that codebook fingerprints are represented as probability distributions, selecting an appropriate similarity metric is critical for accurate learnware retrieval. We systematically evaluate seven metrics: Cosine similarity, Bhattacharyya coefficient [47], Hellinger distance [48], Total Variation, KL divergence [50], Jensen-Shannon divergence [49], and Euclidean distance (details in Tab. XI). The retrieval performance was measured by

TABLE XII
AVERAGE SGCS PERFORMANCE OF LEARNWARE MODEL REPOSITORY WITH VARIOUS DISTRIBUTIONAL SIMILARITY METRICS. CR=1/16 AND FEEDBACK QUANTIZATION BITS $Q=4$.

Similarity Metrics Sampling Range (m ²)	LOS						NLOS					
	5 × 5	5 × 10	5 × 20	5 × 30	5 × 40	5 × 50	5 × 5	5 × 10	5 × 20	5 × 30	5 × 40	5 × 50
Cosine (s_{cos})	0.886	0.855	0.839	0.721	0.659	0.599	0.324	0.318	0.319	0.303	0.285	0.233
Bhattacharyya [47] (s_{Bhatt})	0.883	0.857	0.855	0.746	0.666	0.612	0.327	0.309	0.312	0.287	0.266	0.209
Hellinger [48] (S_{Hell})	0.883	0.857	0.855	0.746	0.666	0.612	0.327	0.309	0.312	0.287	0.266	0.209
Total Variation (S_{TV})	0.884	0.856	0.854	0.721	0.669	0.625	0.324	0.307	0.302	0.300	0.283	0.226
KL [50] (S_{KL})	0.881	0.847	0.855	0.786	0.719	0.657	0.327	0.296	0.318	0.286	0.270	0.209
Jensen-Shannon [49] (S_{JS})	0.883	0.857	0.854	0.746	0.666	0.612	0.327	0.309	0.312	0.292	0.289	0.209
Euclidean (S_{Euc})	0.873	0.856	0.759	0.678	0.634	0.528	0.325	0.316	0.315	0.299	0.278	0.217
Original Model Repository	0.891	0.864	0.855	0.786	0.737	0.667	0.346	0.333	0.330	0.317	0.309	0.261
Upper Bound	0.915	0.881	0.903	0.868	0.844	0.848	0.372	0.356	0.347	0.324	0.334	0.277

average SGCS performance of selected models across various scenarios (6 LOS and 6 NLOS sampling ranges, each with 10 cases), with reference to the Original Model Repository (Top-1 performance, upper bound for Model Repository).

Tab. XII reveals distinct performance patterns. KL divergence achieves optimal accuracy in LOS large-range environments, matching the repository upper bound. However, it suffers catastrophic failure in NLOS diffuse conditions (0.209 at 5×50 versus 0.261 upper bound). In contrast, Cosine similarity maintains stable performance across all NLOS scenarios (0.324–0.233) without extreme degradation. This contrast stems from fundamental differences in handling low-magnitude diffuse distributions. KL divergence’s logarithmic form $\sum p_i \log(p_i/q_i)$ amplifies relative errors when absolute probabilities are small (characteristic of NLOS multipath scattering), causing similar diffuse patterns to be misjudged as dissimilar. Cosine similarity’s L_2 -normalization $\cos \theta = \frac{\mathbf{p}^T \mathbf{q}}{\|\mathbf{p}\| \|\mathbf{q}\|}$ eliminates sensitivity to absolute energy concentration, focusing on relative angular patterns rather than absolute power levels.

We select Cosine similarity for learnware retrieval due to its robust performance across both LOS concentrated and NLOS scattered distributions, combined with computational efficiency (Low complexity versus High for KL), enabling real-time retrieval in large-scale repositories.

5) *Multi-carrier OFDM Scenario*: To verify the extensibility of the proposed framework beyond single-carrier systems, we conduct additional experiments on a multi-carrier OFDM scenario with explicit CSI feedback, where the 512-subcarrier channel is transformed via 2D-DFT to the angle-delay domain and pruned to the first 32 delay taps; the real-valued and normalized version of $\mathbf{H} \in \mathbb{C}^{32 \times 32}$ is then fed into the auto-encoder. Following the same design principle as the angular-domain codebook fingerprint, we construct a dual-domain statistical specification $\mathbf{pdf}_{\text{joint}} = [\mathbf{pdf}_{\text{angle}}; \mathbf{pdf}_{\text{delay}}] \in \mathbb{R}^{64}$. Specifically, for each channel sample, we extract the angular domain by projecting each subcarrier’s channel vector onto the DFT codebook with $B = 5$ and taking a majority vote across all subcarriers to obtain the dominant angular index $\text{idx}_{\text{angle}}$, which filters out subcarrier-level fluctuations and identifies the path consistently observed across the entire bandwidth. For the delay domain, since $\mathbf{H}$ is already in the delay domain, we directly average the energy across antennas to obtain the power delay profile, from which the dominant delay index $\text{idx}_{\text{delay}}$ is

identified. Aggregating these indices across the dataset yields $\mathbf{pdf}_{\text{angle}} = \text{hist}_{\mathcal{D}}(\text{idx}_{\text{angle}})$ and $\mathbf{pdf}_{\text{delay}} = \text{hist}_{\mathcal{D}}(\text{idx}_{\text{delay}})$.

We construct a repository of 200 Learnware models (100 LOS + 100 NLOS) using TF-1 architecture ($L_{\text{TF}} = 2$, $\text{feature_dim}=64$, $\text{Embedding_dim} = 64$), each trained on 10,000 samples. A TF-4 ($L_{\text{TF}} = 12$, $\text{Embedding_dim} = 192$) General Model is trained on a mixture of 2 million samples. Fig. 10(c) presents the performance on representative LOS/NLOS test cases ($5\text{m} \times 5\text{m}$). The selected Learnware consistently outperforms the General Model and approaches the Original Model Repository, confirming the framework’s effectiveness in multi-carrier OFDM systems.

VI. CONCLUSION

This paper proposes a Learnware-based framework for intelligent deployment of CSI feedback models in wireless communication systems. Hosted in a centralized AI data center, the framework supports model reuse across diverse deployment scenarios, reducing training overhead and improving adaptability. To address key limitations of conventional model repositories, such as data privacy concerns, communication costs, and inefficient retrieval, the proposed approach introduces semantic and statistical specifications to enable accurate and lightweight model selection. The Learnware Model Repository eliminates the need for transmitting raw CSI data and exhaustive model evaluations by leveraging codebook fingerprint distributions for retrieval. An adjustable-speed retrieval strategy ensures scalability, while fine-tuning based on a user-defined threshold enables efficient model adaptation. Extensive simulations verify that the Learnware model repository consistently outperforms baseline methods. Compared to the general model with fine-tuning, the Learnware approach achieves higher SGCS performance, with representative gains of 18.8% in LOS and 57.7% in NLOS scenarios, while reducing fine-tuning overhead by 300–1000 samples (depending on propagation conditions) and 100 training epochs. Furthermore, hierarchical LSH model retrieval and complexity analysis confirms that the retrieval latency is negligible, validating the repository’s practicality for real-time deployment.

REFERENCES

- [1] ITU-R WP5D, “Framework and overall objectives of the future development of IMT for 2030 and beyond,” Radio Communication Division of the International Telecommunication Union (ITU-R), Tech. Rep., Jun. 2023. [Online]. Available: <https://www.itu.int/md/R19-WP5D-230612-TD-0905/en>

- [2] S. Dang, O. Amin, B. Shihada, and M.-S. Alouini, "What should 6G be?" *Nature Electron.*, vol. 3, no. 1, pp. 20–29, Jan. 2020.
- [3] A. Alhammad, I. Shayea, A. A. El-Saleh, M. H. Azmi, Z. H. Ismail, L. Kouhalvandi, and S. A. Saad, "Artificial intelligence in 6G wireless networks: Opportunities, applications, and challenges," *Int. J. Intell. Syst.*, vol. 2024, no. 1, p. 8845070, Mar. 2024.
- [4] T. L. Marzetta, "Massive MIMO: an introduction," *Bell Labs Tech. J.*, vol. 20, pp. 11–22, Mar. 2015.
- [5] M. E. Eltayeb, T. Y. Al-Naffouri, and H. R. Bahrami, "Compressive sensing for feedback reduction in MIMO broadcast channels," *IEEE Trans. Commun.*, vol. 62, no. 9, pp. 3209–3222, Sept. 2014.
- [6] M. S. Rahman, Y.-H. Nam, J. Zhang, and J.-Y. Seol, "Linear combination codebook based CSI feedback scheme for FD-MIMO systems," in *Proc. 2015 IEEE Globecom Workshops (GC Wkshps)*, Feb. 2015, pp. 1–6.
- [7] J. Guo, C.-K. Wen, S. Jin, and G. Y. Li, "Overview of deep learning-based CSI feedback in massive MIMO systems," *IEEE Trans. Commun.*, vol. 70, no. 12, pp. 8017–8045, Dec. 2022.
- [8] C.-K. Wen, W.-T. Shih, and S. Jin, "Deep learning for massive MIMO CSI feedback," *IEEE Wirel. Commun. Lett.*, vol. 7, no. 5, pp. 748–751, Oct. 2018.
- [9] Y. An, S. Lu, H. Cai, and Z. Ji, "A deep learning-based approach to lightweight CSI feedback," *Phys. Commun.*, vol. 68, p. 102538, Feb. 2025.
- [10] S. Jiang and A. Alkhateeb, "Digital twin aided massive MIMO: CSI compression and feedback," in *Proc. ICC 2024-IEEE Int. Conf. Commun.*, IEEE, Aug. 2024, pp. 3586–3591.
- [11] J. Chen, W. J. Hillery, K. R. Mestav, C. Nuzman, I. Saniee, and Y. Xing, "CSI compression for massive MIMO: Model-based or data-driven?" *IEEE Wirel. Commun.*, vol. 32, no. 1, pp. 22–27, Feb. 2025.
- [12] X. Liang, Z. Jia, X. Gu, and L. Zhang, "Toward better low-rate deep learning-based CSI feedback: A test channel-based approach," *IEEE Trans. Wirel. Commun.*, vol. 23, no. 8, pp. 8773–8786, Aug. 2024.
- [13] J. Shin, Y. Kang, and Y.-S. Jeon, "Vector quantization for deep-learning-based CSI feedback in massive MIMO systems," *IEEE Wirel. Commun. Lett.*, vol. 13, no. 9, pp. 2382–2386, Sep. 2024.
- [14] Y.-C. Lin, T.-S. Lee, and Z. Ding, "A scalable deep learning framework for dynamic CSI feedback with variable antenna port numbers," *IEEE Trans. Wirel. Commun.*, vol. 23, no. 4, pp. 3102–3116, Apr. 2024.
- [15] Z. Peng, R. Liu, Z. Li, C. Pan, and J. Wang, "Deep learning-based CSI feedback for XL-MIMO systems in the near-field domain," *IEEE Wirel. Commun. Lett.*, vol. 13, no. 12, pp. 3613–3617, Dec. 2024.
- [16] 3GPP TR 38.843, "Study on artificial intelligence (AI)/machine learning (ML) for NR air interface," 3rd Generation Partnership Project (3GPP), Tech. Rep., Dec. 2023. [Online]. Available: https://ftp.3gpp.org/Specs/archive/38_series/38.843/38843-200.zip
- [17] X. Lin, "A tale of two mobile generations: 5G-Advanced and 6G in 3GPP release 20," *IEEE Commun. Stand. Mag.*, pp. 1–9, Oct. 2025.
- [18] J. Hoydis, F. A. Aoudia, A. Valcarce, and H. Viswanathan, "Toward a 6G AI-native air interface," *IEEE Commun. Mag.*, vol. 59, no. 5, pp. 76–81, May 2021.
- [19] N. Ye, S. Miao, J. Pan, Q. Ouyang, X. Li, and X. Hou, "Artificial intelligence for wireless physical-layer technologies (AI4PHY): A comprehensive survey," *IEEE Trans. Cognit. Commun. Netw.*, vol. 10, no. 3, pp. 729–755, Jun. 2024.
- [20] C.-H. Huang, C.-K. Wen, and G. Y. Li, "AI/ML life cycle management for interoperable AI native RAN," *arXiv preprint arXiv:2507.18538*, Jul. 2025.
- [21] J. Guo, Y. Cui, C.-K. Wen, and S. Jin, "Prompt-enabled large AI models for CSI feedback," *IEEE J. Sel. Areas Commun.*, vol. 44, pp. 2654–2668, 2026.
- [22] S. Alikhani, G. Charan, and A. Alkhateeb, "Large wireless model (LWM): A foundation model for wireless channels," *arXiv preprint arXiv:2411.08872*, 2024.
- [23] M. Chen, M. Liu, Z. Zhang, Z. Xu, and L. Wang, "Task-oriented semantic communication with foundation models," *China Commun.*, vol. 21, no. 7, pp. 65–77, Jul. 2024.
- [24] Y. Li, G. Li, Z. Wen, S. Han, S. Gao, G. Liu, and J. Wang, "Channel modeling aided dataset generation for AI-enabled CSI feedback: Advances, challenges, and solutions," *IEEE Commun. Stand. Mag.*, vol. 8, no. 4, pp. 72–78, Dec. 2024.
- [25] M. Inoue, T. Ohtsuki, K. Yamamoto, and G. Gui, "Evaluation of source data selection for DTL based CSI feedback method in FDD massive MIMO systems," in *Proc. 2023 IEEE 20th Consumer Commun. Netw. Conf. (CCNC)*, Mar. 2023, pp. 182–187.
- [26] J. Sun, Y. Zhang, G. Gui, H. Zhao, H. Gacanin, and H. Sari, "Interacting federated and transfer learning-aided CSI prediction for intelligent cellular networks," *IEEE Trans. Veh. Technol.*, vol. 72, no. 12, pp. 15776–15787, Dec. 2023.
- [27] Y. Zhang and A. Alkhateeb, "Zone-specific CSI feedback for massive MIMO: A situation-aware deep learning approach," *IEEE Wirel. Commun. Lett.*, vol. 13, no. 12, pp. 3320–3324, Dec. 2024.
- [28] J. Liu, M. Bouazizi, and T. Ohtsuki, "Source channel model selection for downlink CSI feedback estimation using transfer learning in massive MIMO system," in *Proc. 2025 Int. Conf. Comput., Netw. Commun. (ICNC)*, May 2025, pp. 276–281.
- [29] Z.-H. Zhou, "Learnware: on the future of machine learning," *Frontiers Comput. Sci.*, vol. 10, no. 4, pp. 589–590, Aug. 2016.
- [30] Y.-J. Zhang, Y.-H. Yan, P. Zhao, and Z.-H. Zhou, "Towards enabling learnware to handle unseen jobs," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 35, no. 12, May 2021, pp. 10964–10972.
- [31] Z.-H. Zhou and Z.-H. Tan, "Learnware: Small models do big," *Sci. China Inf. Sci.*, vol. 67, no. 1, p. 112102, Oct. 2024.
- [32] Z.-H. Tan, J.-D. Liu, X.-D. Bi, P. Tan, Q.-C. Zheng, H.-T. Liu, Y. Xie, X.-C. Zou, Y. Yu, and Z.-H. Zhou, "Beimingwu: A learnware dock system," in *Proc. 30th ACM SIGKDD Conf. Knowledge Discovery Data Mining*, Aug. 2024, pp. 5773–5782.
- [33] H.-Y. Lei, Z.-H. Tan, and Z.-H. Zhou, "On the ability of developers' training data preservation of learnware," in *Proc. 38th Advances Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, Curran Associates, Inc., Dec. 2024, pp. 36471–36513.
- [34] M. Chen, J. Guo, C.-K. Wen, S. Jin, G. Y. Li, and A. Yang, "Deep learning-based implicit CSI feedback in massive MIMO," *IEEE Trans. Commun.*, vol. 70, no. 2, pp. 935–950, Feb. 2022.
- [35] M. Baur, N. Turan, S. Wallner, and W. Utschick, "Evaluation metrics and methods for generative models in the wireless PHY layer," *IEEE Trans. Machine Learning Commun. Netw.*, vol. 3, pp. 677–689, May 2025.
- [36] J. Suh, C. Kim, W. Sung, J. So, and S. W. Heo, "Construction of a generalized DFT codebook using channel-adaptive parameters," *IEEE Commun. Lett.*, vol. 21, no. 1, pp. 196–199, Jan. 2017.
- [37] J. Baxter, "A model of inductive bias learning," *J. Artif. Intell. Res. (JAIR)*, vol. 12, pp. 149–198, Mar. 2000.
- [38] A. Maurer, M. Pontil, and B. Romera-Paredes, "The benefit of multitask representation learning," *J. Artif. Intell. Res. (JAIR)*, vol. 17, no. 81, pp. 1–32, 2016.
- [39] K. Dwivedi and G. Roig, "Representation similarity analysis for efficient task taxonomy & transfer learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 12387–12396.
- [40] J. Guo, Y. Cui, S. Jin, and J. Zhang, "Large AI models for wireless physical layer," *IEEE Commun. Mag.*, vol. 64, no. 5, pp. 148–155, May 2026.
- [41] J. Morais, S. Alikhani, A. Malhotra, S. Hamidi-Rad, and A. Alkhateeb, "A dataset similarity evaluation framework for wireless communications and sensing," in *2024 58th Asilomar Conf. Signals Syst. Comput.*, 2024, pp. 1144–1149.
- [42] R. Ahmed, E. Visotsky, and T. Wild, "Explicit CSI feedback design for 5G new radio phase II," in *Proc. 22nd Int. ITG Workshop Smart Antennas (WSA)*, Jun. 2018, pp. 1–5.
- [43] S. Jaekel, L. Raschkowski, K. Börner, and L. Thiele, "QuaDRiGa: A 3-D multi-cell channel model with time evolution for enabling virtual field trials," *IEEE Trans. Antennas Propag.*, vol. 62, no. 6, pp. 3242–3256, Jun. 2014.
- [44] H. Xiao, Z. Wang, D. Li, W. Tian, X. Liu, W. Liu, S. Jin, J. Shen, Z. Zhang, and N. Yang, "AI enlightens wireless communication: A transformer backbone for CSI feedback," *China Commun.*, vol. 21, no. 12, pp. 243–256, Dec. 2024.
- [45] 3GPP, "New SI: Study on artificial intelligence (AI)/machine learning (ML) for NR air interface," 3GPP, Technical Report RP-213599, December 2021, 3GPP TSG RAN Meeting #94e, Electronic Meeting.
- [46] "Technical Specification Group Radio Access Network; NR; Physical layer procedures for data (Release 17)," 3rd Generation Partnership Project (3GPP), Technical Specification 3GPP TS 38.214 V17.0.0, Mar. 2022.
- [47] A. Bhattacharyya, "On a measure of divergence between two statistical populations defined by their probability distributions," *Bull. Calcutta Math. Soc.*, vol. 35, pp. 99–110, 1943.
- [48] E. Hellinger, "Neue Begründung der Theorie quadratischer Formen von unendlichvielen Veränderlichen," *Crelle's J.*, vol. 136, pp. 210–271, 1909.
- [49] J. Lin, "Divergence measures based on the Shannon entropy," *IEEE Trans. Inf. Theory*, vol. 37, no. 1, pp. 145–151, 1991.
- [50] S. Kullback and R. A. Leibler, "On information and sufficiency," *Ann. Math. Statist.*, vol. 22, no. 1, pp. 79–86, 1951.